QueueFlow: Computer Vision Based Automated Queue Ticket Assignment and Real-Time Predictive System for Dynamic Campus Service Delivery

A Capstone Research

Presented to the Faculty of the
College of Computer Studies
Naga College Foundation, Inc.
Naga City

In Partial Fulfillment of
The Requirements for the Degree
Bachelor of Science in Computer Science

Balbin, Archie D.
Sanota, Joshua Rovic P.

2027

NAGA COLLEGE FOUNDATION, INC.
College of Computer Studies
City of Naga

APPROVAL SHEET

This capstone research of **ARCHIE D. BALBIN** and **JOSHUA ROVIC P. SANOTA** entitled **"QUEUEFLOW: COMPUTER VISION BASED AUTOMATED QUEUE TICKET ASSIGNMENT AND REAL-TIME PREDICTIVE SYSTEM FOR DYNAMIC CAMPUS SERVICE DELIVERY"** was prepared and submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science. It has been examined and is recommended for acceptance and approval for oral examination.

Fritzgerald Enric T. Imperial, MIT
Adviser

THESIS COMMITTEE:

DR. ANNA LORETTA C. ROMULO
Chairman

DR. RONEL B. SIMON
Panel Member

RONALD O. BALDEMORO
Panel Member

LEILA B. GARDOSE, MBA
Panel Secretary

PANEL OF EXAMINERS

Approved by the Committee on Pre-Oral Examination on _____.

DR. ANNA LORETTA C. ROMULO
Chairman

DR. RONEL B. SIMON
Panel Member

RONALD O. BALDEMORO
Panel Member

Accepted and approved in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science.

DR. ANNA LORETTA C. ROMULO
Dean

NAGA COLLEGE FOUNDATION, INC.
College of Computer Studies
City of Naga

SECRETARY'S CERTIFICATION

This capstone research of **ARCHIE D. BALBIN** and **JOSHUA ROVIC P. SANOTA** entitled **"QUEUEFLOW: COMPUTER VISION BASED AUTOMATED QUEUE TICKET ASSIGNMENT AND REAL-TIME PREDICTIVE SYSTEM FOR DYNAMIC CAMPUS SERVICE DELIVERY"** was prepared and submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science. It has been examined and is recommended for acceptance and approval for oral examination.

LEILA B. GARDOSE, MBA
Secretary

NAGA COLLEGE FOUNDATION, INC.
College of Computer Studies
City of Naga

EDITOR'S CERTIFICATION

This capstone research of **ARCHIE D. BALBIN** and **JOSHUA ROVIC P. SANOTA** entitled **"QUEUEFLOW: COMPUTER VISION BASED AUTOMATED QUEUE TICKET ASSIGNMENT AND REAL-TIME PREDICTIVE SYSTEM FOR DYNAMIC CAMPUS SERVICE DELIVERY"** was prepared and submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science. It has been examined and is recommended for acceptance and approval for oral examination.

DR. ANNA LORETTA C. ROMULO
Editor

ACKNOWLEDGMENT

The researchers express their sincere gratitude to Fritzgerald Enric T. Imperial, MIT, their adviser, for his guidance, technical advice, and patience throughout the development and writing of this capstone research.

Abstract

Title:	**QueueFlow: Computer Vision Based Automated Queue Ticket Assignment and Real-Time Predictive System for Dynamic Campus Service Delivery**
Researcher:	Archie D. Balbin and Joshua Rovic P. Sanota
Type of Document:	Unpublished Undergraduate Capstone Research (2027)
Host/Accrediting School:	Naga College Foundation, Inc.
City:	Naga City
Region:	Region V
Year of Completion:	2027
Adviser:	Fritzgerald Enric T. Imperial, MIT
Keywords:	Queue management, computer vision, YOLOv8, person re-identification, predictive analytics, FastAPI, printer-agnostic ticket protocol, QR code, JWT, M/M/c model, Holt's double exponential smoothing

Queueing in many campus service offices is still handled by staff observation, manual numbering, and verbal updates at the counter. During enrollment, payment, and clearance periods, students often do not know how far they are from service, while staff must estimate the line from what they can see at the counter. QueueFlow was built as a capstone prototype for that office setting. It combines a YOLOv8n detector, OpenCV camera processing, automatic queue-number assignment, ticket issuance, and short-horizon wait-time forecasting. The prototype runs the camera detector separately from the FastAPI backend: the detector sends tracked-person metadata, duplicate-filtered detections, crowd counts, and annotated snapshots, while the backend confirms entrants inside a calibrated queue zone before issuing a number. Confirmation uses a candidate window, motion checks, track-ID remapping, and a hybrid appearance-spatial re-entry rule so that brief exits from the camera view do not immediately produce duplicate queue tickets. QueueFlow also uses a printer-agnostic ticket payload containing the queue number, QR link, short code, and JSON Web Token (JWT), which can be rendered as a thermal ticket in deployment or as a PDF ticket in the prototype build. Queue records are stored in MySQL, students can check ticket status from a mobile client, and staff can monitor live snapshots, no-show alerts, counter actions, analytics, and forecasts through the dashboard. The forecasting component reports the current estimated wait time and 5-, 15-, and 30-minute projections using an M/M/c baseline, local trend projection, Holt's double exponential smoothing, and a mean-reversion adjustment. The Naga College Foundation, Inc. (NCF) prototype is evaluated through queue assignment accuracy, re-identification reliability, ticket and token validation, prediction error using MAE and MAPE, system

latency, and user satisfaction.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Background of the Study

In a busy campus office, the queue is not only a line of students; it is also an early signal of how much pressure the office is about to handle. During enrollment, payment, clearance, and related transactions, students may arrive faster than staff can manually count, organize, and update them. The result is a familiar but difficult situation: students wait without a clear view of their status, while staff make service, skip, recall, and counter decisions with only partial real-time information.

This problem is not unique to Naga College Foundation, Inc. (NCF). Recent reports from Philippine higher education institutions document the persistence of long enrollment queues, congested service windows, and the operational gap between rising student volumes and manual queue management capacity [1]. At NCF, informal accounts from students and service staff indicate that wait times during enrollment, payment, and clearance periods can become substantial, with congestion most pronounced during the first weeks of each semester. This pattern, combined with the absence of a real-time queue display and the manual recording of transaction order, indicates the same kind of inefficient service delivery that the literature identifies as a recurring issue in Philippine HEIs.

At NCF, congestion also affects the continuity of student transactions. Enrollment, cashiering, and library services are separate offices, but students often experience them as one connected process. A delay at one window can affect the next step that the student needs to complete. The Commission on Higher Education recognizes accessibility and efficient service delivery as indicators of institutional quality [2]; for this reason, improving queue visibility is treated in this study as part of improving the student service experience, not merely as a counter-management task.

Conventional queue systems are useful once a student has requested a number or once staff have encoded the transaction. Their limitation is that they normally begin after the office has already recognized the arrival. They can store issued tickets, but they do not usually observe the service area, confirm new arrivals from a camera view, reduce duplicate queue entries, or estimate near-term changes in the line. The gap is therefore not only ticket issuance; it is the missing link among arrival detection, staff monitoring, student status access, and prediction.

QueueFlow responds to this gap through a camera-connected queue management prototype for campus service offices. Instead of waiting for a manual entry, it uses YOLOv8n and OpenCV to detect persons inside a defined queue zone. YOLO is appropriate for this use case because it performs object detection in a single network pass [3]. YOLOv8n was selected because it is light enough for prototype-grade hardware while still supporting ByteTrack-based persistent tracking [4] and stable deployment tooling [5, 6]. Newer YOLO variants, including YOLOv11 and YOLOv12, introduce architectural improvements [7, 8]; however, this study prioritizes stable single-class person detection in

a controlled queue zone over marginal accuracy gains that may require more computing resources. The selection therefore reflects the constraints of the prototype environment rather than a general claim that YOLOv8n is superior in all settings.

QueueFlow extends detection with duplicate filtering, candidate confirmation, and hybrid re-entry matching so that queue records remain more consistent when people move or briefly leave the camera frame. The ticket component is printer-agnostic: each confirmed queue entry receives a queue number, a short code, a QR link, and a JSON Web Token (JWT) for status checking. At the service counter, this payload can be printed on thermal paper. For the prototype runs, it is rendered as a downloadable PDF, allowing the ticket identity and validation flow to be tested without making the printer hardware the focus of the study.

Beyond ticket generation, QueueFlow connects the detector, queue records, staff dashboard, and prediction module through a FastAPI-based backend and MySQL database. The prediction module reports current, 5-minute, 15-minute, and 30-minute wait-time estimates using queuing logic and short-term forecasting. For NCF, the prototype is intended to answer a concrete operational question: whether a camera-fed queue record can give staff and students a clearer view of the line before a manual counter update is made.

The Review of Related Literature therefore examines the technologies and concepts that support each part of the problem identified above. Computer vision and YOLO literature address automated queue observation, queue management literature explains ticketing and identity maintenance, and forecasting literature supports the wait-time estimates needed for staff decision-making.

1.2 Review of Related Studies

1.2.1 Computer Vision for Real-Time Person Detection

Recent computer-vision work shows why camera-based queue observation is technically feasible. Sindagi and Patel [9] surveyed CNN-based crowd counting and density-estimation methods, noting the move from earlier scale-limited approaches to models that handle occlusion, complex scenes, and dense groups more reliably. One-stage detectors such as YOLO are relevant to this study because they process a frame quickly enough for live monitoring [3]. Terven et al. [10] trace YOLO's development from YOLOv1 through YOLOv8 and YOLO-NAS, with attention to the accuracy-speed tradeoff in each generation. Wang et al. [6] more recently proposed a YOLOv8n-derived person detector for edge-device surveillance, which supports the decision to use a lightweight YOLOv8n base in the QueueFlow prototype.

Because QueueFlow uses a camera, privacy is treated as a design constraint rather than an afterthought. Padilla-Lopez et al. [11] survey visual privacy protection methods such as camera-level anonymization, selective scrambling, and aggregate-only transmission. In QueueFlow, this concern is reflected in the decision to work with bounding boxes and colour histograms instead of storing or transmitting raw video.

1.2.2 YOLO-Based Tracking and Deduplication

YOLO's single-pass detection architecture delivers the speed necessary for real-time queue monitoring, processing entire frames in one forward pass to simultaneously predict bounding boxes, confidence scores, and class labels [3]. The YOLOv8 series advances this through improved multi-scale feature pyramid networks and anchor-free detection heads [5], with the broader YOLO family reviewed in depth by Terven et al. [10]. More recent variants extend this trajectory in different directions: YOLOv11 emphasizes a reduced parameter count and improved feature reuse, while YOLOv12 introduces an attention-centric architecture with Area Attention and Residual ELAN modules [7, 8]. These newer variants improve accuracy and contextual reasoning but at marginal latency cost; for single-class person detection in a fixed queue zone, the stability, tracking integration, and benchmark maturity of YOLOv8n remain practically advantageous.

Beyond single-frame detection, multi-object tracking requires associating detections across frames into consistent identities. Wojke et al. [12] introduced the DeepSORT algorithm, which integrates appearance information with motion-based tracking via Kalman filtering and the Hungarian algorithm, reducing identity switches when targets are temporarily occluded. QueueFlow draws on similar principles in its custom track-identifier remapping logic, complemented by a two-stage deduplication architecture: standard intersection-over-union non-maximum suppression performed internally by ByteTrack during detection, and a separate per-frame duplicate-suppression pass at the queue-tracker level that combines IoU and centroid proximity with re-entry immunity and an appearance-based safeguard. The latter pass is required because YOLO occasionally assigns two distinct track identifiers to the same physical person within the same frame.

1.2.3 Queue Management and Person Re-Identification

Camera-based monitoring of service areas enables passive observation of arrivals without requiring user interaction. The core technical challenge beyond detection is person re-identification: when a tracked individual temporarily disappears from view and reappears, the system must recognize them as the same person and restore their queue position rather than assigning a new number.

Ye et al. [13] survey deep learning approaches to person re-identification, dividing methods into closed-world and open-world settings and categorizing the field along three axes: deep feature representation learning, deep metric learning, and ranking optimization. Building on this foundation, more recent reviews by Wang et al. [14] and Asperti et al. [15] document a shift toward Transformer-based representations, self-supervised and unsupervised methods, and improved robustness to occlusion and pose variation. These analyses underscore that practical Re-ID systems typically combine multiple signals: appearance-based methods using colour histograms or deep feature embeddings, spatial-temporal methods using last known position and elapsed time, and hybrid methods combining both. QueueFlow implements a hybrid re-entry matching approach in this tradition. Within a recency window of 120 seconds, the matcher compares a new detection's split-body HSV appearance signature against the signatures of missing queue

entries using Pearson histogram correlation; if the comparison is inconclusive, a spatial proximity check provides a secondary fallback for the single-missing-person case. This appearance-first ordering reduces identity swaps when two persons exchange positions in the line, a case where a spatial-first rule would assign the wrong queue number. The simplified scheme is appropriate for a single-zone prototype and avoids the additional computational overhead of deep Re-ID embeddings, which are typically necessary only when matching across non-overlapping cameras.

At the application layer, Concepcion et al. [1] describe a Smart Queuing and Appointment Management System for Bulacan State University, demonstrating that web-based automated queue management and online scheduling in a Philippine HEI can reduce wait times and improve perceived service quality. That system, however, operates through user-initiated web ticketing and appointment booking rather than passive computer-vision detection of arrivals, which is the gap QueueFlow specifically targets.

1.2.4 Predictive Analytics for Service Demand Forecasting

Queue staff need two kinds of forecast: a current estimate for the person already in line and a short look-ahead for counter planning. Shmueli and Koppius [16] describe predictive analytics as distinct from explanatory modelling because its test is whether empirical models produce useful predictions for decisions.

The M/M/c queuing model provides analytical wait-time calculations from real-time arrival and service rate measurements, making it suitable for current wait-time estimates and near-term tactical predictions [17]. Together with Little's Law, the M/M/c framework gives QueueFlow the mathematical basis for translating live arrival rates into expected waiting times. System utilization is expressed as:

$$\rho = \frac{\lambda}{c\mu} \tag{1.1}$$

where λ is the arrival rate, μ the service rate, and c the number of active servers. The mean number in the system together with Little's Law yields the expected wait time used by the dashboard:

$$W = \frac{L}{\lambda} \tag{1.2}$$

For medium-term forecasting, Holt's double exponential smoothing tracks both queue level and trend with separate smoothing parameters [18]. Hyndman and Athanasopoulos [19] discuss exponential smoothing and multi-horizon forecasting, including damping factors that limit over-projection during temporary spikes. That point matters in campus offices, where a line may grow quickly during enrollment and then thin out once a batch of students is served. For the 30 min horizon implemented in QueueFlow, mean-reversion logic is used to limit long-forecast uncertainty.

1.3 Synthesis of the State-of-the-Art

The reviewed studies point to a practical limitation of ordinary queue records: an issued ticket list does not show what is happening in the service area before a ticket is

printed. Camera-based person detection can fill part of that gap by capturing activity as it occurs. YOLO-based approaches are especially relevant here because their speed allows a prototype to observe students entering and moving through a defined queue zone without waiting for manual encoding.

Within the YOLO family, the trajectory from YOLOv8 through YOLOv11 and the attention-centric YOLOv12 [8] indicates clear architectural progress, but also a widening gap between research benchmarks and the operational constraints of campus prototypes. YOLOv8n was selected for QueueFlow because it offers (a) stable production tooling and Ultralytics-supported persistent tracking, (b) lightweight inference suitable for the available prototype hardware, (c) broad community benchmarking that simplifies reproducibility, and (d) accuracy that is competitive with newer variants for the specific task of single-class person detection in a fixed queue zone. This rationale, rather than version recency, justifies the design choice.

Detection alone is not enough for the QueueFlow use case. A person detector can identify bounding boxes, but it does not decide whether a box represents a new entrant, a duplicate track, a returning student, or a person who should keep an existing queue number. For this reason, tracking, deduplication, candidate confirmation, and re-entry matching are treated as queue-management logic rather than optional post-processing.

Research on queue management and predictive analytics further reinforces the need to move beyond static ticket numbering. Traditional queue tools can arrange students in order, but they provide limited support for staff decisions when demand changes quickly. Queuing theory and short-term forecasting address this limitation by translating arrival rates, active counters, and queue length into wait-time estimates. Multi-horizon prediction is therefore useful because staff need both immediate queue status and near-future indicators for counter planning.

For this manuscript, the design implication is narrow and direct: QueueFlow must connect four functions that are often treated separately, including camera-based queue entry, queue identity management, student ticket/status access, and staff-facing wait-time estimates. The prototype is evaluated on that integration rather than on detection accuracy alone.

Despite these advances, existing studies remain limited when viewed against the specific needs of NCF's campus service context. Many focus on crowd counting, general queue applications, or prediction models as separate components. The Philippine HEI work that does exist, such as Concepcion et al. [1], addresses web-based ticketing and appointment-booking systems rather than passive vision-based detection. Fewer works demonstrate how these components can operate together in a Philippine educational institution where queue visibility, staff action, student status checking, and practical deployment constraints must be handled in a single prototype.

1.4 Gap Bridged by the Study

The literature reviewed in Section 1.2 shows strong work on individual parts of the problem: queue monitoring, object detection, re-identification, and wait-time forecasting. What remains less developed is a campus-service prototype that joins these parts into

one operating workflow for a Philippine higher education institution. Existing Philippine HEI work, including the system reported by Concepcion et al. [1], is centered on web-based ticketing and appointment booking. It does not begin with passive camera-based recognition of arrivals in the service area.

QueueFlow narrows this gap by connecting the arrival, identity, ticket, and prediction tasks inside one system architecture. The prototype uses YOLOv8n person detection together with ByteTrack [4] persistent tracking, augmented by a five-stage quality-gate pipeline at detection time and a post-frame duplicate-suppression layer at the queue-tracker level to keep duplicate queue entries to a minimum. For cases where a person leaves and later returns within a 120-second recency window, a hybrid re-entry matcher compares appearance histograms first and falls back to spatial proximity if appearance is inconclusive. Two safeguards, a twin / lookalike guard that blocks restoration when a candidate resembles a currently visible person, and a done-person blacklist that prevents a served person from receiving a second queue number, protect the matcher's decisions.

The same workflow also prepares the ticket record used by students to check their place in line. It contains the queue number, short code, QR link, and JWT-backed validation record. In the target office setup this record is printed through a thermal printer; during evaluation it is rendered as a PDF ticket so the queue and validation logic can be tested consistently. Finally, the dashboard reports the current wait estimate together with 5-, 15-, and 30-minute forecasts. The contribution is therefore the practical integration of detection, queue identity, ticket validation, and prediction for students and administrative staff in an NCF service office setting.

1.5 Theoretical Framework

The theoretical framework of this study is anchored on the queue visibility problem described in the Background of the Study. Campus service offices need a way to observe student arrival, maintain queue order, estimate waiting time, and present useful information to both staff and students. QueueFlow addresses this problem through the combined application of queuing theory, computer vision, predictive analytics, and forecasting theory.

Queuing Theory, as formalized in classical queueing literature [17], explains how arrivals, service capacity, and waiting lines behave through the M/M/c model. The M/M/c model represents a multi-counter service environment in which the arrival rate, service rate, and number of active counters influence queue length, utilization, and waiting time. In QueueFlow, these equations are tied to queue records, active counter settings, and service-time estimates so the dashboard can convert live counts into wait-time and utilization values.

Computer Vision Theory explains the automated observation layer of the prototype. In QueueFlow, YOLOv8n and OpenCV process camera frames and return person detections inside a defined queue zone. YOLO's single-pass detection approach supports live monitoring because it can identify objects quickly within each frame [3]. This is the first step before queue numbers, tickets, and wait estimates are created.

Predictive Analytics Theory covers the shift from simply showing the current line

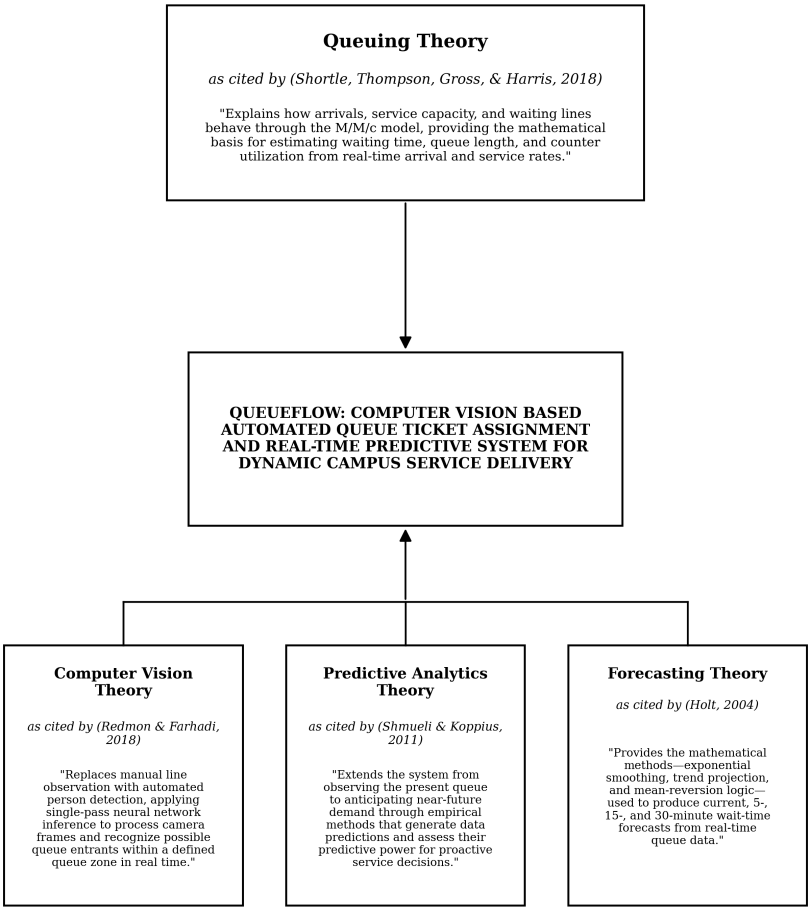

Figure 1: Theoretical Paradigm

Figure 1.1: Theoretical Framework of QueueFlow

to estimating what the line may look like shortly afterward. Shmueli and Koppius [16] frame predictive analytics around empirical prediction and decision support. In QueueFlow, this idea is implemented through the wait-time prediction module used by staff on the dashboard.

Forecasting Theory supplies the specific mathematical methods used for the prediction module. QueueFlow applies exponential smoothing, trend projection, and mean-reversion logic, methods rooted in the classical exponentially weighted moving-average framework [18] and discussed by Hyndman and Athanasopoulos [19], to produce current, 5-minute, 15-minute, and 30-minute wait-time estimates. The medium-horizon forecast in particular relies on Holt's double-exponential smoothing, in which a level L_t and a trend T_t are updated jointly from the observed queue count y_t:

$$L_t = \alpha_{\mathrm{H}}\, y_t + (1 - \alpha_{\mathrm{H}})(L_{t-1} + T_{t-1}), \tag{1.3}$$

$$T_t = \beta_{\mathrm{H}}(L_t - L_{t-1}) + (1 - \beta_{\mathrm{H}})T_{t-1}. \tag{1.4}$$

Smoothing factors α_{H} and β_{H} govern the responsiveness of the level and trend components respectively; their values, together with the damping factor that prevents trend over-projection over the projection horizon, are specified in Section 3.2.9 (§3.2.9.5). These forecasts support staff decision-making by giving the dashboard more than a static queue count; they provide an early indication of whether congestion is likely to increase or decrease.

In this study, each theory is tied to a specific part of the prototype. Computer vision is used to observe queue activity from camera frames, while queuing theory connects the observed queue length with counter capacity and expected waiting time. Predictive analytics turns these counts into dashboard information that staff can act on, and forecasting theory guides the short-term wait-time estimates. These outputs appear in the staff dashboard, the student status page, and the ticketing process. For office use, the ticket record is sent to a thermal printer. For evaluation, the prototype renders it as a PDF while keeping the same queue number, short code, QR link, and JWT.

1.6 Conceptual Framework

The conceptual framework follows the Input-Process-Output-Outcome (IPOO) model. For this prototype, the input side consists of the live camera feed, the marked queue zone, available counters, staff actions, and service-time assumptions. These inputs pass through the main QueueFlow routines: person detection, queue-zone filtering, re-entry checking, ticket creation, and wait-time prediction. The processed results are then shown on the staff dashboard, student status page, and public display board, while queue and ticket records are saved for review.

Figure 1.2 shows how those inputs, processing steps, outputs, outcomes, and feedback loops are arranged in the proposed QueueFlow workflow.

Input. The initial phase consists of the data, settings, and operational conditions required by the system. These include the live camera feed, queue-zone settings, counter configuration, service-time assumptions, staff actions, and the ticket and status-access requirements used by students and administrative staff.

Process. The process stage transforms the input data into queue intelligence through the QueueFlow pipeline. Key processes include YOLOv8n person detection with ByteTrack [4] persistent tracking, five-stage detection quality gating, exponential moving-average smoothing of bounding boxes, zone-based centroid filtering, candidate accumulation over a minimum confirmation window, track-identifier remapping, per-frame duplicate suppression with re-entry immunity and an appearance guard, hybrid appearance-first re-entry matching with twin-guard and done-blacklist safeguards, queue-state management, no-show detection, sticky counter assignment, dynamic service-time measurement, multi-horizon wait-time prediction, and ticket payload generation.

Output. The main outputs are a real-time queue list with assigned queue numbers, ticket credentials that can be printed or rendered as a PDF, individual queue status

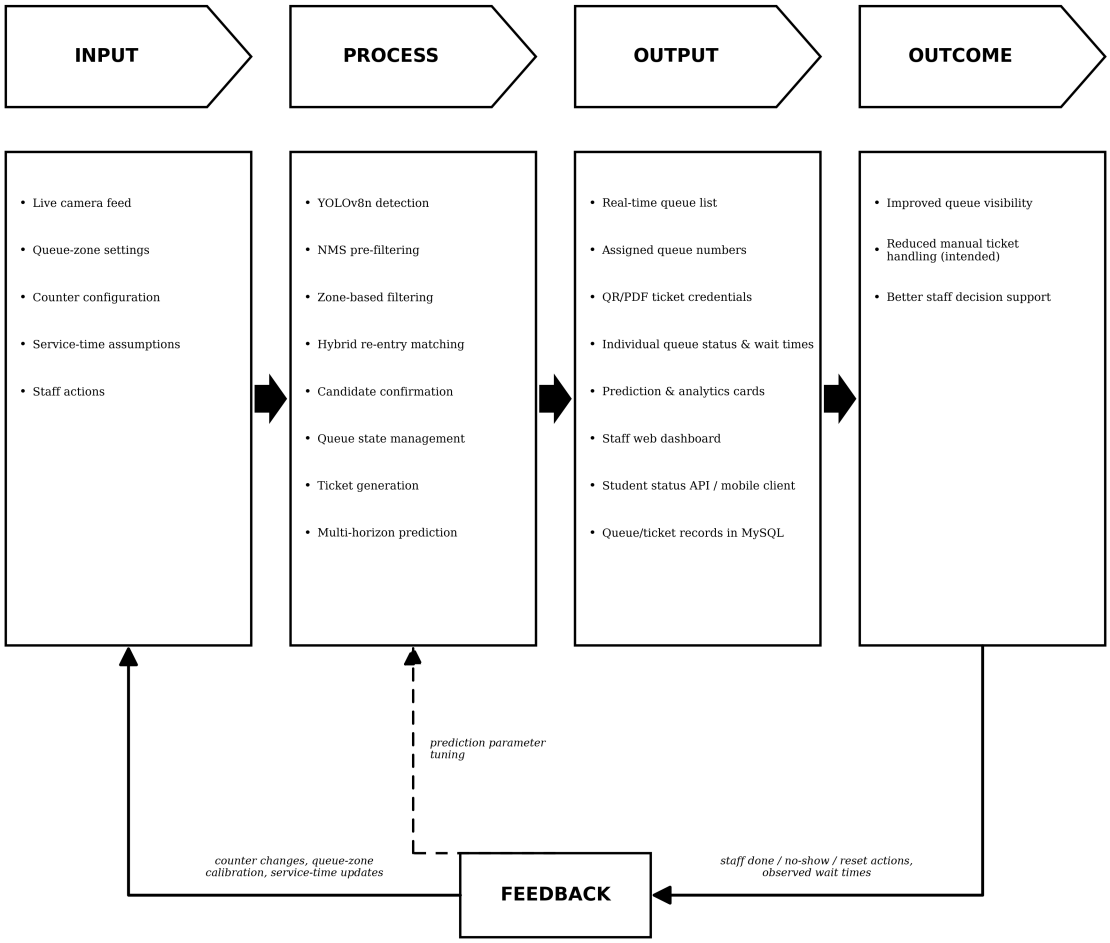

Figure 1.2: Conceptual Framework of QueueFlow

and wait times, prediction and analytics cards, the staff web dashboard, the student status API consumed by a mobile client, and a public display board that announces newly assigned counter calls using the browser's built-in Web Speech API. Queue entries, ticket records, and operational events are also persisted in the MySQL database.

Outcome. The intended outcome is a queue process that staff can see and act on earlier, with fewer manual ticket-handling steps once the thermal-printer deployment is used.

Feedback. Observed wait times together with staff done, no-show, and reset actions are routed into the Feedback stage. This feedback propagates counter changes, queue-zone calibration, and measured service-time updates back into the Input stage. A secondary signal, shown as a dashed arrow in Figure 1.2, feeds prediction-parameter tuning back into the Process stage, allowing the candidate confirmation window, smoothing factors, and mean-reversion thresholds to be adjusted iteratively. This refinement allows QueueFlow to adapt to the actual demand patterns of NCF service offices rather than remaining locked to its initial settings.

1.7 Statement of the Problem

The study is guided by the following research problems:

1. What are the current limitations of the manual queue management process at NCF service offices during peak demand periods, as established through baseline observation of wait times, ticketing practices, and reported student and staff experiences?

2. How effectively can a YOLO-based passive detection pipeline with hybrid re-identification assign and maintain queue numbers without student interaction, as measured by queue assignment accuracy, re-identification reliability, ticket and token validation, and detection-to-ticket latency?

3. To what degree of accuracy, measured by Mean Absolute Error (MAE) and Mean Absolute Percentage Error (MAPE), can the implemented prediction models forecast current, 5-, 15-, and 30-minute wait times?

4. How do service staff and administrators evaluate QueueFlow across the five dimensions measured by the structured Likert-scale instrument used in this study, namely, usability, detection reliability, prediction usefulness, dashboard clarity, and overall satisfaction, and what overall weighted mean does each dimension obtain?

1.8 Assumptions

The following assumptions delimit the proposed evaluation:

Camera visibility and queue-zone coverage. The camera hardware provides sufficient video quality and coverage of the queue zone to enable reliable YOLO person detection under the lighting conditions and pedestrian densities typical of NCF service areas.

Effectiveness of tracking and re-identification. YOLO's tracking mechanism, supplemented by the system's spatial and appearance-based re-identification layer, produces sufficiently stable person identities across the queue duration to keep duplicate queue assignments to an acceptable rate, to be reported empirically in Chapter 3.

Usefulness of prediction outputs. The implemented prediction models generate forecasts that are actionable and sufficiently accurate for staff scheduling and counter management decisions, with accuracy assessed using MAE and MAPE in Chapter 3.

Ticket rendering. The ticket record remains the same whether it is printed on thermal paper or rendered as a PDF: both outputs carry the queue number, short code, QR link, and JWT used for status checking.

Staff capability and system adoption. Service staff and administrators possess basic digital literacy to operate the web dashboard and interpret the system's outputs.

1.9 Scope and Delimitation

This study is limited to the development and evaluation of a camera-assisted queue prototype for one NCF service-office setting in Naga City, Philippines. The build includes the YOLOv8n detection pipeline, hybrid re-entry matching, short- and medium-horizon wait-time prediction, FastAPI endpoints, MySQL storage, a staff dashboard, student status access, and printer-agnostic ticket generation. During office use, the ticket record is meant to be printed through a thermal printer. During evaluation, it is rendered as a PDF so the queue number, short code, QR link, and JWT validation can be checked without relying on printer availability.

The present build uses one camera-defined queue zone and one selected service office. It excludes multi-zone and multi-department operation, and no priority queue is implemented in the system; all detected clients are assigned under the ordinary queue order without priority handling based on student status or urgency. The build also excludes direct links to the Campus Management System or Student Information System and IoT inputs other than the camera feed and ticket printer. The evaluation is limited to one semester after implementation.

1.10 Significance of the Study

The value of the study is in how the prototype supports everyday service-office work: it gives staff a clearer view of the line, provides students with a way to check their queue status, and keeps records that administrators can review when improving service flow.

Service Staff. Service staff receive a dashboard that shows the active queue, individual wait times, counter assignments, and short-horizon demand forecasts without requiring them to recount the line manually.

Students. Students benefit from a passive queueing process that does not require manual registration. Upon entering the monitored area, a queue number is assigned and

can be checked through the status link or mobile client using the ticket credential.

Institutional Administrators. Administrators receive queue performance records that can be used when reviewing staffing, service hours, and physical layout.

Naga College Foundation, Inc. The institution benefits from a campus-oriented prototype that demonstrates how automation, computer vision, and predictive analytics can support service delivery and student-facing operations.

Academic Community and Future Researchers. The work provides a local reference for combining computer vision, re-identification, ticket generation, and predictive analytics in Philippine HEI service queues. Future researchers may extend it to multi-camera zones, priority queues, direct campus-system integration, or alternative detection and forecasting models.

1.11 Definition of Terms

Access Token. A ticket credential assigned to each confirmed queue number. In the implemented prototype, this consists of a short code and a JWT-backed validation record that allows the student to check queue status without a login account.

ByteTrack. The multi-object tracking algorithm [4] invoked by Ultralytics persistent tracking. ByteTrack associates detection boxes across frames using high- and low-confidence detection matching, allowing the system to maintain a stable track identifier tid for each person as they move through the queue zone.

Candidate Accumulation. A buffer mechanism requiring a new detection to appear consistently for a minimum confirmation window of 20 frames, with a centroid-standard-deviation motion check, before a queue number is assigned. The mechanism prevents shadows, momentary mis-detections, and stationary mis-classified objects from generating spurious tickets. The motion gate and confidence bypass are specified in Section 3.2.3 (§3.2.3).

Done Blacklist. A set of appearance signatures retained for served persons during a session, used to suppress new queue entries from a person who has already been served. It is triggered when the Pearson histogram correlation between an incoming detection and a blacklist entry exceeds $\tau_{\text{bl}} = 0.55$.

Duplicate Suppression. A per-frame queue-tracker pass that merges two distinct track identifiers assigned to the same physical person, retaining the lower queue number. It is guarded by a 60-frame re-entry immunity counter and an appearance-similarity safeguard that prevents visually distinct persons from being merged regardless of geometric proximity. The method is specified in Section 3.2.5 (§3.2.5).

Dynamic Service-Time Measurement. A background process that recomputes the average transaction service time every five minutes from inter-departure gaps in the MySQL transaction log, blended seventy–thirty with the previously stored value. It is specified in Section 3.2.10 (§3.2.10).

Exponential Moving Average (Bounding-Box Smoothing). A per-frame smoothing pass applied independently to each bounding-box coordinate using factor $\alpha_{\text{s}} = 0.45$, producing the visually stable boxes rendered in the staff dashboard's annotated snapshot.

Holt's Double Exponential Smoothing. A classical forecasting algorithm [18] used for the medium-horizon prediction, applying separate smoothing parameters for queue level and trend, together with a damping factor to avoid over-projection. Parameter values used in the prototype are configurable defaults reported and justified in Chapters 2 and 3.

M/M/c Model. A foundational multi-server queueing model from classical queueing theory [17] used for short-term wait-time prediction, calculating $W = L/\lambda$ from Little's Law using real-time arrival rates from the YOLO detection pipeline.

Mobile Client. The student-facing interface used to check queue status. It is implemented as a mobile-optimized web page consumed through the student status API; references to a "mobile tracker" or "mobile browser" elsewhere refer to the same component.

Non-Maximum Suppression (NMS). A standard intersection-over-union pass applied internally by ByteTrack during detection to suppress overlapping bounding-box candidates. A separate per-frame duplicate-suppression step at the queue-tracker level, specified in Section 3.2.5 (§3.2.5), handles the related but distinct case of two track identifiers being assigned to the same physical person within the same frame.

Pearson Histogram Correlation. The appearance comparison metric used by re-entry matching, computed independently on the upper- and lower-body halves of the split appearance signature and combined with a sixty–forty weighting in favour of the upper body.

Printer-Agnostic Ticket Protocol. The set of fields, including queue number, short code, QR link, and JWT, issued for each confirmed queue entry. Because the ticket record is independent of the output device, it can be printed on thermal paper in office use or rendered as a PDF during evaluation.

Public Display Board. A user-facing surface, alongside the staff web dashboard and the student mobile client, that renders the active queue in the waiting area and announces newly assigned counter calls using the browser's built-in Web Speech API. All text-to-speech synthesis runs client-side; no audio is transmitted to any external service.

Queue Zone. A configurable rectangular region defined by four pixel coordinates on the camera frame; only persons whose bounding box centre falls within this region are tracked and assigned queue numbers.

Re-Entry Matching. A hybrid algorithm that, within a 120-second recency eligibility window, first compares a new detection's split-body HSV appearance histogram against the histograms of missing queue persons using Pearson correlation. In the single-missing-person case, a spatial-proximity check provides a secondary fallback if the appearance comparison is inconclusive. The full decision rule, including the multi-missing-person branch and the supporting twin-guard and done-blacklist safeguards, is specified in Section 3.2.6 (§3.2.6).

Sticky Counter Assignment. A scheduling rule under which a counter, once allocated to a queue entry, persists until that entry is marked done or no-show, regardless of changes to the line behind it. The rule prevents the disorienting per-frame reshuffling that a straight-rank assignment would otherwise produce.

Twin / Lookalike Guard. A safeguard that blocks re-identification restoration when the appearance similarity between a new detection and a currently visible queue

person exceeds $\tau_{\text{twin}} = 0.85$, on the grounds that two persons with near-identical clothing should each receive their own queue number.

You Only Look Once (YOLO). The YOLOv8n model [5] with ByteTrack-based `persist=True` tracking, configurable input size, and confidence threshold, used to detect and track persons entering the queue zone in camera frames. Newer variants, YOLOv9 through YOLOv12 [8], are not used in this study; the rationale is discussed in Section 1.3.

2 Methodology

Chapter 2 sets out how the QueueFlow prototype will be built, deployed, and evaluated in the selected NCF service office. The chapter covers the research design, respondents and locale, sampling procedure, data gathering, research instruments, questionnaire validation, ethical safeguards, statistical treatment, system architecture, development process, and testing plan.

2.1 Research Design

The research design has two linked parts. The developmental part covers the construction of QueueFlow; the evaluation part checks how the prototype behaves during service-office use. Quantitative evidence comes from system logs, observer records, and survey ratings. These data are used to measure queue assignment accuracy, re-identification reliability, ticket and token validation, detection-to-ticket latency, and wait-time prediction error through Mean Absolute Error (MAE) and Mean Absolute Percentage Error (MAPE). The five-point Likert survey records how service staff, administrators, and student users rate dashboard usability, layout clarity, and the usefulness of the prediction outputs after regular exposure to the prototype. Qualitative evidence comes from short semi-structured interviews, where respondents can explain concerns or useful features that a rating scale may not capture.

The engineering work is organized around the prototype components that carry the main technical risk: YOLOv8n detection, hybrid re-entry matching, ticket-payload generation, multi-horizon prediction, the FastAPI backend, the staff dashboard, and the mobile status interface. Each component is built through the Agile sprint cycle described in Section 2.9 and illustrated in Figure 2.3. Its output is checked against the design principles in Section 2.8. Quantitative results and qualitative accounts are analyzed side by side so that the final discussion can compare measured performance with the actual experience of the users.

2.2 Population and Locale of the Study

The research site is Naga College Foundation, Inc. (NCF) in Naga City, Camarines Sur, Philippines. NCF is an appropriate setting because student transactions in enrollment, cashiering, and library-related services create visible peak periods where manual queue handling is placed under pressure. The prototype will be installed in one service office area chosen with NCF administration. That office serves as the single-queue-zone test site defined in the Scope and Delimitation of Chapter 1.

Three respondent groups are included because they encounter the queue from different positions. Service staff represent the counter-level users who receive students and process transactions. Administrators and IT personnel represent the group responsible

for oversight, deployment, and service-office policy. Currently enrolled NCF students represent the queue participants who receive a ticket and check their status during the evaluation period.

2.3 Sample and Sampling Technique

Sampling is adapted to the size and role of each respondent group rather than applying one procedure to all participants. This allows the evaluation to include the operational perspective of staff, the management perspective of administrators, and the user experience of students.

Service staff. Because the population of staff directly involved in queue-based transactions in the selected service office is relatively small and bounded, total enumeration is used; all eligible staff will be invited to participate. The inclusion criterion is at least two weeks of regular interaction with the system in their normal work role during the evaluation period.

Administrators. Total enumeration is likewise used for this group, since the population of administrators and IT personnel with oversight of the selected service office is small. The inclusion criterion is direct supervisory involvement with the service office during the evaluation period.

Students. Because the student population is large, the sample size for this group is determined using Slovin's formula at a 5% margin of error:

$$n = \frac{N}{1 + Ne^2} \tag{2.1}$$

where n is the required sample size, N is the population of NCF students who interact with the targeted service office during the evaluation period, and e is the margin of error, set at 0.05. Student respondents will then be drawn through stratified purposive sampling across year levels to ensure that the resulting sample reflects the diversity of users who experience the queue. The inclusion criterion is at least one completed transaction involving a QueueFlow-issued ticket during the evaluation period.

Exclusion criterion. Across all three groups, any prospective respondent below 18 years of age is excluded. The age criterion is enforced through a screening question that immediately precedes the informed consent form, and any prospective respondent indicating an age below 18 is thanked for their interest and not enrolled.

2.4 Data Gathering Procedure

Data gathering is divided into three phases so that the manual baseline, live system behavior, and user feedback are not mixed together prematurely. The sequence also makes it possible to compare what staff normally do, what QueueFlow records during use, and what participants report after using the prototype.

Phase 1: Pre-Deployment Setup and Operational Baseline. Prior to deployment, the researchers will coordinate with the office head of the selected service

office to confirm the camera placement, queue zone calibration, counter configuration, and the initial seed value for the average service time. The seed value is used by the prediction service until enough transactions have been logged for the dynamic service-time measurement procedure (Section 3.2.10, §3.2.10) to supply a measured estimate. During this phase, the researchers will also conduct informal observation of the existing manual queue handling process, recording approximate queue lengths and qualitative accounts from staff regarding the patterns of congestion most commonly encountered. These baseline accounts establish the qualitative reference against which the QueueFlow-mediated process is later compared.

Phase 2: System Deployment and Performance Logging. After alpha and beta testing in Section 2.10, the system will be deployed for regular operation in the selected service office for the evaluation period. During this phase, two parallel streams of data will be captured. First, the system will automatically log every detection event, queue assignment, re-entry matching decision, ticket issuance, prediction output, and dashboard interaction, with associated timestamps. Second, an independent observer will maintain a manual ground-truth log during defined observation windows, recording each actual queue entrant, each genuine re-entry event, and the actual wait time measured from queue zone entry to counter arrival. The automated logs and the manual ground-truth log together support the technical performance metrics specified in Section 2.7.

Phase 3: Post-Evaluation Survey and Interviews. After the evaluation period, the validated questionnaire in Section 2.5 will be administered to the three respondent groups identified in Section 2.3. A subset of respondents from each group will be invited to a short semi-structured interview, scheduled at their convenience and lasting approximately twenty minutes. Interview audio will be recorded only with consent and transcribed, after which the recordings will be deleted. Both survey responses and interview transcripts will be anonymized before analysis.

2.5 Research Instrument

Four instruments provide the evidence for the study: the QueueFlow operational log, the observer's ground-truth log, a researcher-made Likert questionnaire, and a short semi-structured interview guide.

Operational and ground-truth logs. The operational log is generated automatically by the system and records every detection, queue assignment, re-entry decision, ticket issuance, prediction output, dashboard action, and associated timestamps. The manual ground-truth log is a paper or digital record maintained by an independent observer during defined observation windows in Phase 2. Together these two logs provide the data needed to compute queue assignment accuracy, re-identification reliability, MAE and MAPE for wait-time prediction, detection-to-ticket latency, and ticket and token validation rates.

Likert-scale questionnaire. The questionnaire is structured into five dimensions that correspond directly to Statement of the Problem No. 4 in Chapter 1: usability of the dashboard, mobile client, and ticket workflow; perceived detection reliability of the queue entry process; perceived usefulness of the current, 5-, 15-, and 30-minute wait-time

predictions; clarity of the dashboard layout, live snapshot, analytics cards, and counter controls; and overall satisfaction with the system. Each dimension contains several items rated on the five-point scale shown in Table 2.1.

Semi-structured interview guide. A short interview guide is prepared to explore usability problems, perceived benefits of the prediction outputs, situations in which staff would override system suggestions, and recommendations for future improvement. The guide is used flexibly, with the interviewer free to follow up on respondent comments.

Table 2.1: Likert Scale Interpretation

Scale	Category	WM Range	Interpretation
5	Strongly Agree	4.21–5.00	Strongly Agree / Excellent
4	Agree	3.41–4.20	Agree / Very Satisfactory
3	Neutral	2.61–3.40	Neutral / Satisfactory
2	Disagree	1.81–2.60	Disagree / Poor
1	Strongly Disagree	1.00–1.80	Strongly Disagree / Very Poor

Instrument validation. Because the questionnaire is researcher-made, its content validity and reliability are established before it is used in the main evaluation. Content validation is conducted by three expert raters, namely the thesis adviser, one faculty member with expertise in research methods or statistics, and one faculty member with expertise in computer science or information systems. Each rater independently rates every item on a four-point relevance scale, where 1 means not relevant and 4 means highly relevant. The Content Validity Index is then computed at the item level (I-CVI) and at the scale level (S-CVI/Average). An I-CVI of at least 0.78 per item and an S-CVI/Average of at least 0.90 for the full scale are taken as acceptable thresholds, and any item falling below the threshold is revised or removed based on the experts' written comments before pilot testing.

Reliability testing. After content validation, the questionnaire is pilot-tested with approximately 10 to 15 respondents drawn from the same population strata but excluded from the final sample. Internal-consistency reliability is then computed using Cronbach's alpha, with the formula given in Section 2.7 and a value of at least 0.70 taken as acceptable. If alpha falls below the threshold, items with low item-total correlation are revised and the pilot is repeated. The signed content validation forms and the pilot reliability output are attached as appendices.

2.6 Ethical Considerations

Ethical handling is treated as part of the system design, not only as a research requirement. Staff, administrators, and student users may join surveys, interviews, and system testing only after giving informed consent. Before any questionnaire or interview is administered, the researchers explain the purpose of the study, the activities involved, the expected time commitment, possible benefits, and the participant's right to withdraw without penalty.

Only respondents who are 18 years of age or older will be enrolled. The age criterion is enforced through a screening question that immediately precedes the consent form, and any prospective respondent indicating an age below 18 will not be enrolled. No data from minors is collected or stored at any point in the study.

For confidentiality, personal identifiers are removed before analysis. Questionnaires use respondent codes instead of names. Interview audio, when the respondent allows recording, is transcribed for analysis and then deleted. Results are reported in aggregate form so that individual respondents are not identifiable. Digital files are kept in password-protected storage accessible only to the researchers and the thesis adviser, while hard-copy materials are placed in locked storage and shredded after the retention period. Study data are kept for a maximum of three years after completion and are then destroyed.

Camera Snapshot Handling. The system captures live video frames from the queue zone solely for automated person detection and the live staff dashboard. The detector encodes annotated snapshots, showing the camera frame with queue zone, person bounding boxes, and queue numbers overlaid, at a configurable frame rate and transmits them to the backend through an authenticated endpoint. The backend retains only the latest snapshot in process memory; each new snapshot overwrites the previous one, and no historical snapshot archive is maintained on disk. Where an optional Redis cache is enabled in cloud deployment, it is configured as a volatile cache only: snapshot keys carry a short time-to-live, with a default of ten seconds, and Redis persistence, including RDB snapshotting and AOF append-only logging, is disabled in the bundled Redis configuration (`save ""; appendonly no`). If a managed Redis provider is used, persistence and provider-side backups for the QueueFlow cache instance must be disabled, or the existence of provider-side persistence must be disclosed in the deployment notes. Detection metadata, including person tracks, queue length, and wait-time predictions, is transmitted through a separate endpoint and is the only data persisted in the MySQL database; the snapshot image itself is not written to MySQL at any point. The system does not perform facial recognition and does not store biometric templates. Re-identification relies on colour histograms, or clothing-colour distributions derived from bounding-box regions, which are non-biometric and are reset at end-of-session.

Access Control and Transparency. Dashboard access is restricted to authenticated staff through session login, with JWT-based camera-token validation between the detector and the backend. The student-facing mobile client exposes only the requesting student's own queue status, accessed through the ticket QR code and short-code credential issued to that student; it does not display the camera feed or any other student's status. Physical signage will be posted at the service-office entrance informing visitors that the area is monitored by an automated queue management system for service-delivery purposes, citing the Data Privacy Act of 2012, or Republic Act No. 10173, and providing researcher contact for data-subject requests. The camera is mounted visibly, not concealed. Consistent with the proportionality principle of the Act, only the minimum data necessary for queue management is processed; no marketing, profiling, or third-party data sharing is performed.

The work is aligned with the Data Privacy Act of 2012, or Republic Act No. 10173, and with applicable NCF research ethics protocols. Ethical clearance will be secured from

the host institution before deployment and data collection begin. Sources are cited and acknowledged throughout the manuscript. Any use of AI-assisted tools is limited to draft organization, language checking, or proofreading; the system implementation, analysis, interpretation, and final responsibility for the manuscript remain with the researchers.

2.7 Statistical Treatment of Data

The statistical treatment is organized around the questions raised in Chapter 1. Technical measures are used for the detector, ticket, and prediction components, while descriptive statistics are used for the respondent ratings and profile data.

Queue Assignment Accuracy and Re-Identification Reliability. These two metrics address Statement of the Problem No. 2. Queue assignment accuracy is computed as the proportion of correctly assigned queue numbers, matching the manual ground-truth log, over the total number of arrivals observed during the evaluation period. Re-identification reliability is computed as the proportion of correct re-entry decisions, either correctly restoring a previous queue number or correctly assigning a new one, over the total number of re-entry events observed:

$$\text{Accuracy} = \frac{\text{correct decisions}}{\text{total decisions}} \times 100\% \tag{2.2}$$

Mean Absolute Error and Mean Absolute Percentage Error. These two metrics address Statement of the Problem No. 3 and are computed separately for the current, 5-minute, 15-minute, and 30-minute prediction horizons:

$$\text{MAE} = \frac{1}{n} \sum |y_{\text{predicted}} - y_{\text{actual}}| \tag{2.3}$$

$$\text{MAPE} = \frac{1}{n} \sum \left| \frac{y_{\text{actual}} - y_{\text{predicted}}}{y_{\text{actual}}} \right| \times 100\% \tag{2.4}$$

where $y_{\text{predicted}}$ is the system's wait-time estimate at a given horizon and y_{actual} is the wait time measured by the observer using a stopwatch from queue zone entry to counter arrival.

Weighted Mean and Overall Weighted Mean. These two measures address Statement of the Problem No. 4. The weighted mean (WM) is computed for each Likert item, each dimension averaged across items within a dimension, and each respondent group: service staff, administrators, and students. The results are then pooled into an overall weighted mean (OWM). Per-group weighted means are reported to allow comparison across stakeholder perspectives.

$$\text{WM} = \frac{\sum(f \times w)}{N} \tag{2.5}$$

where f is the frequency of each response, w is the weight assigned to that response from 1 to 5 on the Likert scale, and N is the number of respondents in the group.

$$\text{OWM} = \frac{\sum \text{WM}}{n} \tag{2.6}$$

where n is the number of items per dimension or the number of dimensions per group, depending on which level of aggregation is being computed. Interpretation follows the verbal categories in Table 2.1.

Percentage Distribution. Percentage is used to describe the composition of the respondent sample by group, year level, role, and demographic profile, and the distribution of responses across the five-point scale for each item:

$$P = \frac{f}{N} \times 100 \tag{2.7}$$

Cronbach's Alpha. Cronbach's alpha is used to assess the internal-consistency reliability of the questionnaire after the pilot test:

$$\alpha = \frac{k}{k-1}\left(1 - \frac{\sum \sigma_i^2}{\sigma_{\text{total}}^2}\right) \tag{2.8}$$

where k is the number of items, σ_i^2 is the variance of each item, and σ_{total}^2 is the variance of the total score. Values of 0.70 or higher are taken as acceptable.

Detection-to-Ticket Latency. Detection-to-ticket latency is reported as the mean and standard deviation, in seconds, of the elapsed time from the first confirmed detection of a queue entrant to the issuance of the corresponding ticket payload, computed from the automated operational log.

2.8 System Design and Architecture

QueueFlow is designed as a set of cooperating components rather than as a single monolithic application. The detector, re-identification logic, ticket workflow, prediction module, dashboard, FastAPI backend, and MySQL data layer are developed and tested iteratively so that changes in one component can be checked against the behavior of the rest of the prototype.

2.8.1 Design Principles

Detection Accuracy. The YOLOv8n detection pipeline, supplemented by the custom non-maximum-suppression pre-filter, candidate confirmation window, and hybrid re-entry matching logic, must meet the queue assignment accuracy and re-identification reliability thresholds reported in Chapter 3. These are the primary quality gates before deployment to live operation.

Prediction Usefulness. The prediction module must produce wait-time estimates whose MAE and MAPE, at each of the current, 5-minute, 15-minute, and 30-minute horizons, are low enough to be actionable for staff planning. Specific target thresholds are reported in Chapter 3.

API Reliability. The REST endpoints must handle malformed, empty, or unusually long inputs gracefully, returning structured error responses rather than crashing. Endpoint behavior must be consistent and deterministic across repeated calls with identical inputs, and inference time must remain within bounded latency for live use.

Reproducibility. All model configurations, queue-zone parameters, prediction smoothing factors, and confirmation-window values used in the prototype are recorded as configurable defaults in version-controlled code stored in the project repository, ensuring results reported in this study can be independently verified.

Privacy Protection. Consistent with the visual privacy literature reviewed in Chapter 1, the system processes bounding-box and colour-histogram data rather than storing or transmitting raw video. Personally identifiable images are not retained beyond the live frame buffer required for detection.

2.8.2 Tools and Technologies

The system uses Python as the principal implementation language, with FastAPI as the web framework, OpenCV for camera handling, the Ultralytics implementation of YOLOv8n for detection and persistent tracking, custom Python modules for queue-zone filtering, hybrid re-entry matching, and prediction, and MySQL as the data store. Table 2.2 summarizes the tools and their rationale.

Table 2.2: System Tools and Technologies

Category	**Technology**	**Rationale**
Detection	YOLOv8n (Ultralytics)	Lightweight person detection with ByteTrack-based persistent tracking [4]; rationale for choosing this variant over YOLOv9–v12 is given in Chapter 1.
Backend	Python (FastAPI)	REST API, staff dashboard routes, detector upload endpoints, MJPEG streaming, and student status endpoints.
Video	OpenCV (cv2)	Camera capture, frame preprocessing, annotation, and JPEG encoding for dashboard snapshots.
Tracking	Custom Python	Queue-zone filtering, NMS pre-filter for duplicate head/body box suppression, track-ID remapping, hybrid re-entry matching, and no-show handling.
Prediction	Custom Python	M/M/c wait-time baseline, short trend projection, Holt's double exponential smoothing, and mean-reversion long forecast.
Ticketing	Thermal printer; PDF output	Ticket record containing the queue number, short code, QR link, and JWT. Thermal printing is used for office operation; PDF output is used during evaluation.
Database	MySQL	Queue records, short codes, JWTs, no-show events, prediction snapshots, and transaction history.
Auth	Session/JWT	Staff session login, camera-token validation between detector and backend, and student ticket tokens.
Dev Tools	VS Code, Postman, Git/GitHub	Standard development and version control tooling; Postman is used for endpoint contract testing.

2.8.3 System Architecture

The QueueFlow system follows a six-layer architecture composed of the Presentation Layer, the API Layer, the Computer Vision Layer, the Business Logic Layer, the Persistence Layer, and the Cache and Runtime State Layer. This separation follows the implemented project structure: user-facing screens are kept apart from API coordination, camera processing, queue services, database storage, and runtime state management. Figure 2.1 summarizes the layers and their components.

Presentation Layer. This layer comprises three user-facing surfaces. The *staff web dashboard* presents queue analytics, live snapshots or video streaming, counter controls, and no-show alerts. The *student mobile application* supports QR scanning and queue-status display through the ticket short code and JSON Web Token issued at queue entry. The *public display board*, mounted in the service-office waiting area, renders the active queue and announces newly assigned counter calls using the browser's built-in Web Speech API; all text-to-speech synthesis runs client-side and no audio leaves the browser, in keeping with the privacy posture established in Section 2.6 (§2.6).

API Layer. This layer is implemented in Python with FastAPI in `app/main.py`. The backend is composed of five routers: `pages.py` serves the login and dashboard pages and the live MJPEG video stream; `detector_api.py` receives the two endpoints used by the detector process, `POST /yolo/push-frame` for detection metadata and `POST /yolo/update` for annotated snapshots; `queue.py` serves the queue list, queue status, prediction, analytics, and staff queue actions; `crowd.py` serves snapshot retrieval and recent history; and `health.py` reports backend, database, cache, and snapshot status.

Computer Vision Layer. This layer is implemented as a separate Python process in `app/detector.py` running on the camera-connected machine. It opens the camera through OpenCV `VideoCapture`, runs the Ultralytics YOLOv8n detector with ByteTrack [4] persistent tracking, and applies a five-stage quality-gate pipeline to each detection: minimum bounding-box area, maximum frame-coverage fraction, portrait aspect ratio, motion energy, and standard non-maximum suppression internal to ByteTrack. Detection bounding boxes are smoothed across frames with an exponential moving average to remove per-frame jitter before they are forwarded. The detector draws queue overlays such as queue zone, person boxes, queue numbers, and counter assignment indicators onto the frame, and uploads the resulting metadata and JPEG snapshot to the backend through the two detector endpoints. Calls are authenticated by a camera token transmitted in the `X-CAM-TOKEN` header. Running the detector as a separate process prevents camera processing from blocking dashboard or API requests. Quality-gate thresholds and the smoothing factor are specified in Section 3.2.2 (§3.2.2).

Business Logic Layer. This layer is implemented in `app/services/` and contains the queue, prediction, counter, and ticketing services. The *Queue Service* filters incoming detections by removing weak detections, applies the centroid-in-rectangle zone membership test (§3.2.3), and routes confirmed detections into the Queue Tracker. The *Queue Tracker* maintains the active queue in memory; performs candidate confirmation through a 20-frame buffer with a motion gate (§3.2.3); assigns queue numbers to confirmed entrants; performs track-identifier remapping (§3.2.4) and per-frame duplicate suppression with re-entry immunity and an appearance-similarity safeguard (§3.2.5); per-

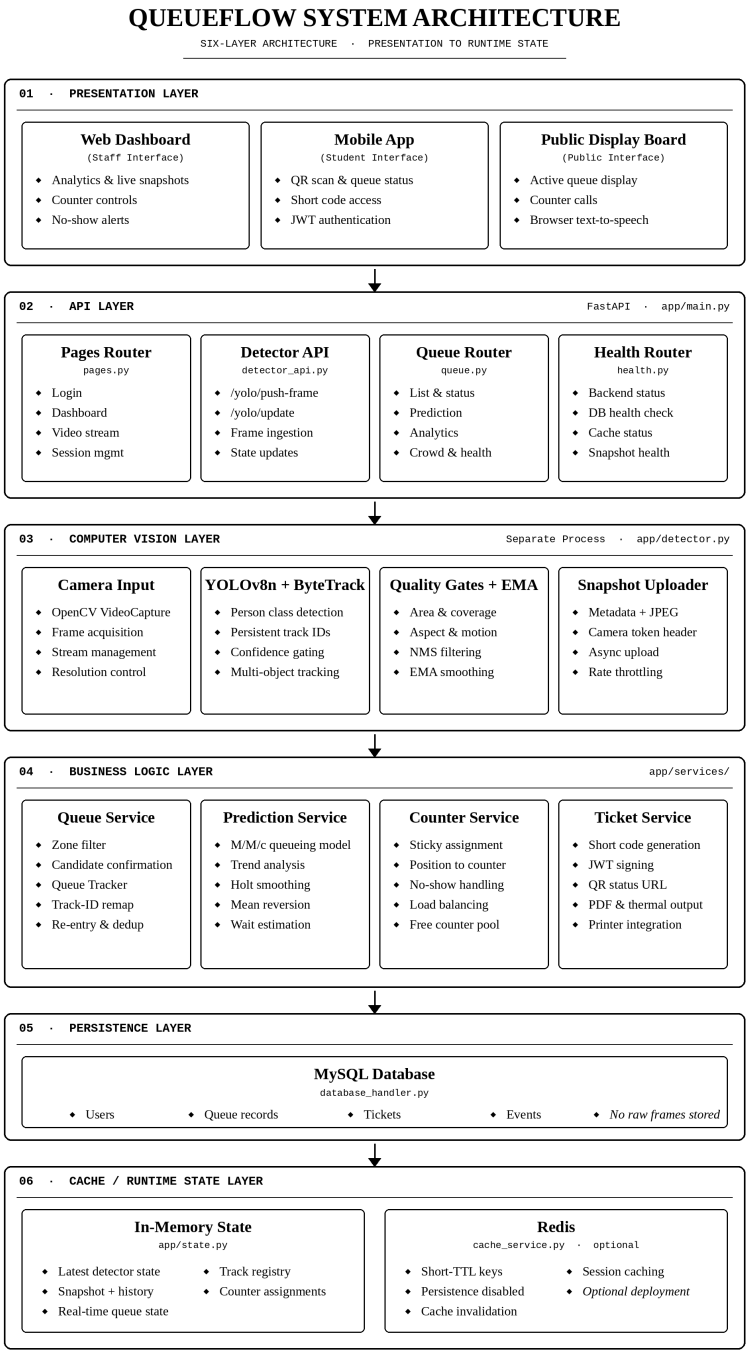

Figure 2.1: QueueFlow Six-Layer System Architecture

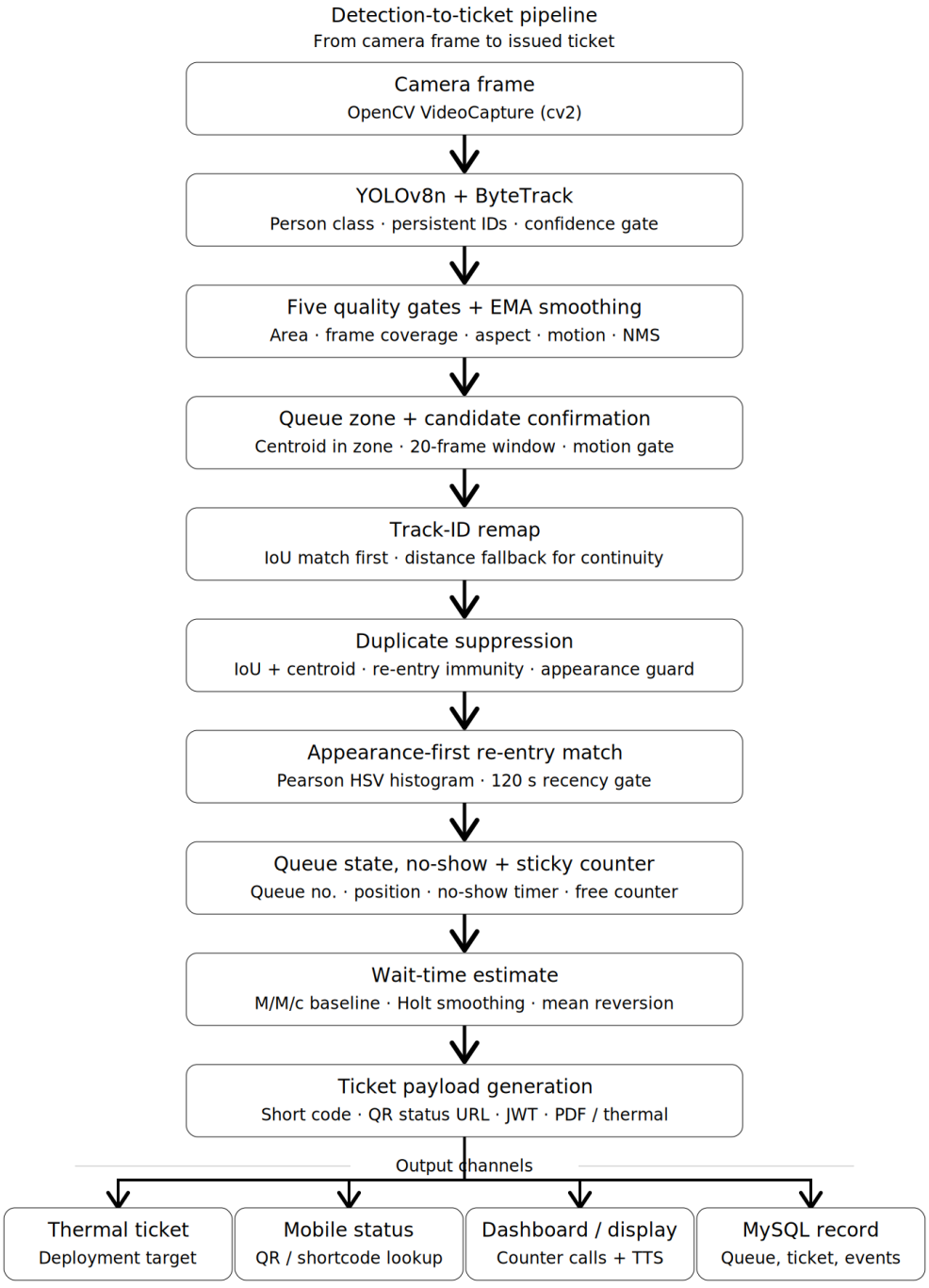

Figure 2.2: QueueFlow Detection-to-Ticket Pipeline

forms hybrid appearance-first re-entry matching with twin-guard and done-blacklist safeguards (§3.2.6); detects no-shows by countdown for the position-one entry (§3.2.7); and produces the sticky counter assignments displayed on the dashboard (§3.2.8). Per-frame bounding boxes are forwarded already smoothed by the Computer Vision Layer.

The *Prediction Service* runs four sub-models in sequence on a rolling buffer of crowd-count samples: an M/M/c queueing baseline for the current wait estimate (§3.2.9.1); a linear trend-projection forecast at the five-minute horizon (§3.2.9.4); Holt's double-exponential smoothing forecast at the fifteen-minute horizon (§3.2.9.5); and a growth-ratio mean-reversion forecast at the thirty-minute horizon (§3.2.9.6). The arrival rate λ is estimated from the difference of recent queue counts (§3.2.9.2); the service rate μ is supplied by the dynamic service-time measurement procedure described in Section 3.2.10 (§3.2.10), which recomputes the average service time every five minutes from the MySQL transaction log.

The *Counter Service* allocates counters to queue entries using a sticky-assignment rule: each frame, the active queue is sorted by queue number, each person receives a position-in-line, and each unassigned person at a position less than or equal to the configured number of counters is given the smallest currently free counter number; allocations persist until the entry is marked done or no-show. The full specification is in Section 3.2.8 (§3.2.8).

The *Ticket Service* is a background worker that consumes ticket-generation jobs, and the *Ticket Printer* produces the short code, the JSON Web Token signed with HMAC-SHA256, the QR-encoded status URL, and the PDF ticket or, in the deployment configuration, drives the thermal printer. The short code is drawn from a 32-character unambiguous alphabet, excluding 0, O, 1, and I, and formatted as `XXXX--XXXX`. The JWT carries a four-hour expiry and a 32-hex-digit nonce to prevent replay. The complete payload specification is in Section 3.2.11 (§3.2.11).

Persistence Layer. This layer is implemented in `app/database/database_handler.py` against a MySQL database. Persisted data include users, queue records such as queue number, short code, JWT token, PDF path, status, expiration, creation timestamp, and serving timestamp, and operational events. Raw camera frames, live JPEG snapshots, and the in-memory queue tracker state are intentionally not persisted to MySQL.

Cache and Runtime State Layer. This layer comprises an in-memory state module in `app/state.py` that holds the latest detector state, the latest annotated snapshot, and a rolling history, and an optional Redis cache in `app/services/cache_service.py` that mirrors these values for cloud deployments where multiple backend workers may need to share live data. The Redis cache is configured as volatile only: keys carry short time-to-live values, and Redis persistence, including RDB and AOF, is disabled in the bundled Redis configuration so that snapshots are never written to disk. If a managed Redis provider is used, persistence and backups must be disabled for the QueueFlow cache instance or disclosed in the deployment notes.

2.8.4 Dynamic Service-Time Measurement

QueueFlow treats average service time as a value that can change during live operation, not as a fixed assumption decided before deployment. Every five minutes, a

background measurement routine reads recent completed transactions from the MySQL log and estimates the current service pace from inter-departure gaps. For completed transaction timestamps $t_1, t_2, \ldots, t_N$, each gap is computed in minutes as

$$g_i = \frac{t_i - t_{i-1}}{60}, \qquad i = 2, \ldots, N. \tag{2.9}$$

Gaps below 0.1 minute are treated as database-write noise, while gaps above 15 minutes are treated as idle periods rather than normal cashier service. The retained gaps are averaged and multiplied by the number of active counters to account for parallel service. The new estimate is then blended with the previously stored value:

$$\widehat{\mathrm{svc}}_{\mathrm{new}} = 0.7\,\widehat{\mathrm{svc}} + 0.3\,\widehat{\mathrm{svc}}_{\mathrm{prev}}. \tag{2.10}$$

This dynamic measurement supplies the service rate μ used by the prediction module after the system has recorded enough transactions. The pre-deployment service-time calibration in Section 2.4 is therefore used only as the initial seed value.

2.8.5 Machine-Learning Wait-Time Predictor Training Protocol

QueueFlow does not train its wait-time model while the cashiering prototype is already serving users. Training is handled as an offline step: the researchers prepare the transaction logs, test several predictors on past records, and deploy the selected model only after its error values are acceptable for the study. This keeps experimental model changes away from live queue operations and gives the researchers a reproducible basis for the results presented later in the manuscript.

Dataset. The working dataset comes from historical cashiering queue logs made available for QueueFlow predictor development. The XML export contains 49,303 queue records covering March 24, 2023 to May 4, 2026. The available fields include queue identifier, application identifier, queue number, service date, service time, transaction type, processing status, and program identifier. Names, student numbers, payment amounts, account details, and other personally identifiable fields are not used by the model. Records marked with `processing = 3` are excluded because they represent no-show, cancelled, or incomplete queue events.

Feature Engineering. The predictor uses three feature groups. The first group is derived directly from the queue logs and timestamp sequence:

1. Queue and transaction fields: queue number, transaction type, program identifier, daily queue size, and estimated people ahead.
2. Time fields: hour, minute, day of week, month, morning or afternoon indicator, and peak-hour indicator.
3. Recent-demand fields: arrivals within the previous 5 and 15 minutes.

The second group consists of academic-period indicators for enrollment, prelim, midterm, final, and clearance periods. These indicators allow the model to distinguish ordinary service days from predictable cashiering-demand spikes.

The third group grounds the AI model in the analytical prediction framework of QueueFlow. For every training row, the preparation script computes outputs from the mathematical routines described in Section 3.2.9: an M/M/c-style baseline wait, a local trend-projected wait, a Holt double-exponential smoothing estimate, a growth-ratio mean-reversion estimate, the estimated arrival rate, and the estimated service time. These analytical outputs are used as input features to the machine-learning model rather than as replacements for it.

The response variable is reconstructed wait time in minutes. Because the XML export does not contain both a queue-entry timestamp and a counter-arrival timestamp for every record, wait time is reconstructed as the elapsed time from a valid queue record to the next valid served record on the same service date. Reconstructed waits below one minute are excluded during model fitting because sub-minute gaps may represent immediate transaction encoding or log artifacts rather than meaningful queue waiting. Reconstructed waits above 120 minutes are excluded to avoid idle-period outliers.

Train and Test Split. Queue records are split by service date, not by random row assignment. The first 80% of ordered service dates form the training set and the remaining 20% form the test set. This order matters because cashiering demand is time-dependent: enrollment week, payment deadlines, and ordinary class days do not behave the same way. A random split could allow future queue patterns to influence model tuning, so it is avoided.

Model Architecture. The implemented AI component uses Gradient Boosting Regression through scikit-learn's `HistGradientBoostingRegressor`. Gradient boosting is selected because the available data is tabular and contains mixed operational, calendar, and analytical prediction features. The model learns nonlinear corrections from historical queue behavior while still being anchored to the interpretable mathematical estimates computed by the prediction service.

Deep sequence models such as Long Short-Term Memory networks and the Temporal Fusion Transformer are not used in this version of the prototype. These architectures may be useful for large, multi-office time-series datasets, but the available cashiering data comes from a single service office. For this study, simpler tabular models are more appropriate and easier to validate.

Hyperparameter Selection. The prototype model uses 300 boosting iterations, a learning rate of 0.05, 31 maximum leaf nodes, and L2 regularization of 0.1. These conservative settings were chosen to keep the model stable on the reconstructed wait-time target and to avoid overfitting to short timestamp gaps.

Evaluation Protocol. The selected model is evaluated on the date-based held-out test set using MAE, RMSE, MAPE, and R^2. MAE and RMSE are treated as the primary metrics because reconstructed waits can be short, making percentage-based error unstable. MAPE is still reported for transparency but is interpreted cautiously.

Integration Plan. After offline testing, the trained model is serialized with `joblib` and loaded by the FastAPI backend at startup. During a ticket-status request, the backend computes the live feature vector, including queue-state features, academic-period features, and the analytical prediction features described above. The Gradient Boosting prediction is then calibrated with current-day service speed, yesterday's same-hour wait behavior, and recent prediction error before being returned to the dashboard

or student status page as the displayed wait-time estimate.

Retraining and Model Drift. The procedure in this subsection covers the initial training run only. Periodic retraining, automated drift monitoring, and production retraining pipelines are outside the present scope, although the prototype is arranged so that a later model file can replace the original one.

Training Result. After cleaning and wait-time reconstruction, the prepared dataset contained 30,362 supervised rows. Applying the one-minute minimum-wait filter left 19,119 modeling rows. The date-based split produced 13,031 training rows and 6,088 test rows. The training period covered May 24, 2023 to January 20, 2026, and the test period covered January 21, 2026 to May 4, 2026. On the test set, the Gradient Boosting model obtained MAE = 2.3512 minutes, RMSE = 4.0858 minutes, MAPE = 80.74%, and $R^2 = -0.0349$. The MAE and RMSE are used as the main accuracy indicators, while MAPE is reported with caution because short reconstructed waits inflate percentage error.

2.9 Software Development Process

QueueFlow development is organized as a small Agile-style engineering cycle. Requirements, interface decisions, coding work, testing, and review are repeated across modules instead of being handled as one large build at the end of the study.

Planning and Requirement Analysis. The researchers will collaborate with NCF service office staff to determine system requirements. User needs, expected workflows, dashboard actions, and ticket conventions will be documented to create user stories, prioritize features, and align development goals with the conceptual framework presented in Chapter 1.

System Design. This phase focuses on the architecture and design of the detection pipeline, the backend, the database, and the dashboard. It includes queue-zone coordinate design, ticket-payload schema design, prediction-module parameter design, and dashboard wireframing.

Development Iteration and Sprint Cycles. Development is organized into sprint cycles, each concentrating on a specific module. The sprints, in the order in which they will be executed, are summarized below:

1. YOLOv8n pipeline setup, queue-zone configuration, and the custom NMS pre-filter for duplicate head and body box suppression.

2. Hybrid re-entry matching algorithm design, implementation, and iterative debugging using recorded test footage.

3. No-show detection and queue position recalculation logic.

4. Prediction module implementation of the M/M/c baseline, short-term trend projection, Holt smoothing, and mean-reversion long forecast; iterative accuracy tuning against logged ground-truth wait times.

5. FastAPI backend, staff web dashboard, and counter controls.

6. Printer-agnostic ticket workflow, including JWT issuance, short-code generation, QR linking, and PDF rendering for demonstration.

7. Student status integration, including mobile client, status API, and end-to-end ticket scan tests.

8. Integration testing, performance tuning, and final pre-evaluation hardening.

Figure 2.3 summarizes the per-sprint cycle through which each of the eight sprints listed above is executed. Sprint-level Plan and Design refine the module's acceptance criteria and interface contracts before Build; Test validates the deliverable against those criteria; and Review captures adviser feedback together with measured performance, which then propagates into the Plan phase of the following sprint. This per-sprint feedback loop instantiates, at the development level, the Feedback stage described in the conceptual framework of Section 1.6.

Each sprint will conclude with a review against defined acceptance criteria and with the thesis adviser before proceeding to the next phase. Sprint reviews will also be used to refine prediction parameters and queue-zone calibration based on observed system behavior, consistent with the feedback loop described in Section 1.6 of Chapter 1.

2.10 Validation and Testing Methods

Validation combines software tests, observed performance, and user feedback. Functional and integration tests check whether the prototype works as intended; queue assignment, re-identification, prediction, and latency measures address the technical research questions; and the survey and interviews address user evaluation.

Alpha Testing. Alpha testing will be conducted internally by the developers using both live camera input and pre-recorded test footage. Defects in detection, tracking, ticket generation, dashboard rendering, and mobile-client status retrieval will be identified and resolved before any external user is exposed to the system.

Beta Testing. The system will be released to a limited group of NCF service staff for at least two weeks of regular operation. During this period, the developers will observe real-world interaction patterns, collect informal feedback, and log any operational issues. Issues affecting the user experience will be addressed before the structured evaluation begins.

Performance Testing: Queue Assignment Accuracy. During defined observation windows, an independent observer will maintain a manual ground-truth log in which every actual queue entrant will be recorded with a timestamp. The system's queue assignments will then be compared against this log. Accuracy is reported as the proportion of correct assignments over total arrivals, using the formula in Section 2.7.

Performance Testing: Re-Identification Reliability. Re-entry events, in which a tracked person leaves the camera frame and later returns, will also be recorded in the same manual ground-truth log. The system's re-entry decision, either assigning a new number or restoring a previous number, will be compared against the ground-truth

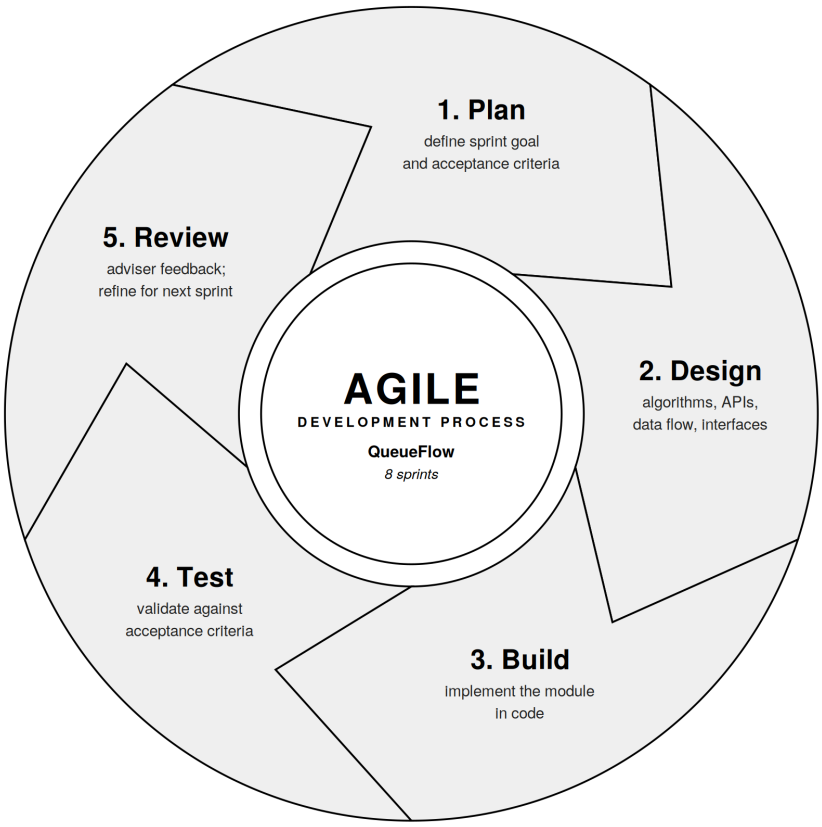

Figure 2.3

Figure 2.3: Agile Development Process for QueueFlow. Each of the eight sprints enumerated above completes one revolution of this five-phase cycle, Plan, Design, Build, Test, Review, before the next sprint begins. The Review phase routes adviser feedback and measured performance back into the Plan phase of the following sprint, instantiating, at the development level, the feedback loop described in Section 1.6.

identity recorded by the observer. Reliability is reported as the proportion of correct decisions over total re-entry events.

Performance Testing: Prediction MAE and MAPE. Actual wait times will be measured by the observer using a stopwatch from the moment a person enters the queue zone to the moment the person reaches the counter. These observed wait times will be compared to the system's predictions at the corresponding horizons. MAE and MAPE will be computed using the formulas in Section 2.7, separately for the current, 5-minute, 15-minute, and 30-minute horizons.

Performance Testing: Latency and Ticket Flow. Detection-to-ticket latency will be measured from automated timestamps logged at the moment a queue entrant is first detected, that is, the moment the detection passes all five quality gates of the Computer Vision Layer (§3.2.2), to the moment the corresponding ticket payload is issued after candidate confirmation and queue-number assignment. Ticket flow validation will be performed by issuing tickets across a representative range of conditions and verifying that the short code, QR link, and JWT each correctly resolves to the matching queue status. Failures will be logged and counted toward the ticket and token validation rate reported under Statement of the Problem No. 2.

Usability Evaluation. After a minimum of two weeks of regular system use, the content-validated and pilot-tested questionnaire in Section 2.5 will be administered to the three respondent groups identified in Section 2.3. Responses will be analyzed using the statistical procedures in Section 2.7. A subset of respondents from each group will be invited to a short semi-structured interview, and the resulting transcripts will be thematically coded to surface patterns that the Likert ratings alone do not capture.

The combination of automated performance logs, manual ground-truth observation, structured survey responses, and qualitative interviews provides multiple lines of evidence for each research question, supporting the convergent parallel mixed-methods design described in Section 2.1.

3 Results and Discussions

3.1 Results

3.2 Design and System Plans

Section 3.2 records the design routines behind the implemented QueueFlow prototype. Section 2.8.3 (§2.8.3) identifies the major layers and responsibilities; this section moves one level closer to the code by stating each routine's inputs, outputs, equations, parameter values, and pseudocode when the control flow would otherwise be hard to follow. The discussion follows the detection-to-ticket flow in Figure 2.2 (Figure 2.2): camera-frame filtering, queue-zone confirmation, tracker repair, re-identification, queue-state updates, prediction, ticket output, public display output, and the parameter summary. Pseudocode blocks are placed after the equations and guard conditions they use so that each algorithm stays with its own design step.

3.2.1 Notation

The specifications below adopt the following notation. Queueing-theoretic symbols (λ, μ, c, ρ, W) retain the meanings introduced in Section 1.5 (§1.5). Queue length is denoted Q throughout this section, consistent with the algorithmic literature; Q is identical to L in Equation 1.2. Two distinct smoothing factors appear and must not be confused: α_{H} and β_{H} for Holt's double-exponential smoothing of queue counts (§3.2.9), and α_{s} for the per-frame exponential moving average applied to bounding-box coordinates (§3.2.2). Thresholds are denoted τ with a descriptive subscript (τ_{tb} for the appearance tiebreak, τ_{twin} for the twin guard, τ_{bl} for the done blacklist, τ_{iou} for the duplicate-suppression IoU gate). The intersection-over-union of two axis-aligned bounding boxes A and B is

$$\mathrm{IoU}(A, B) \;=\; \frac{|A \cap B|}{|A \cup B|}, \tag{3.1}$$

used by the duplicate-suppression step (§3.2.5), the track-identifier remapping step (§3.2.4), and the non-maximum suppression step internal to the tracker (§3.2.2).

3.2.2 Computer Vision Layer

The Computer Vision Layer runs as a separate Python process (`app/detector.py`) on the camera-connected machine. Each captured frame is fed to a YOLOv8n detector [5] configured for single-class person detection. Multi-object tracking is performed by ByteTrack [4], the default tracker when the Ultralytics interface is invoked with `persist=True`; ByteTrack assigns a persistent track identifier tid to each detected person

across frames. Raw YOLO output is then passed through five quality gates before any detection is forwarded to the Business Logic Layer.

3.2.2.1 Detection quality gates

For a bounding box with corners (x_1, y_1, x_2, y_2) in a frame of size $W_f \times H_f$, the five gates are defined as follows. Gate 1 requires sufficient area:

$$A \;=\; (x_2 - x_1)(y_2 - y_1) \;\geq\; A_{\min}. \tag{3.2}$$

Gate 2 caps the fraction of the frame a single box may occupy:

$$f \;=\; \frac{A}{W_f\, H_f} \;\leq\; f_{\max}. \tag{3.3}$$

Gate 3 enforces a portrait-orientation aspect ratio, which removes most false positives caused by long horizontal objects:

$$r \;=\; \frac{y_2 - y_1}{x_2 - x_1} \;\geq\; r_{\min}. \tag{3.4}$$

Gate 4 requires evidence of motion, computed as the number of pixels whose grey-level difference between consecutive frames exceeds a threshold:

$$M \;=\; \big|\{p : |g_t(p) - g_{t-1}(p)| > 25\}\big|. \tag{3.5}$$

A detection passes Gate 4 if $M \geq M_{\min}$ or, as an override for static high-confidence detections, if the detector confidence satisfies conf $\geq c_{\text{byp}}$. Gate 5 is standard non-maximum suppression (NMS) applied internally by ByteTrack using Equation (3.1). Only detections that pass all five gates reach the queue tracker. Parameter values are listed in Table 3.1.

Algorithm 1 Detection quality filtering

Require: current frame F_t, previous grey frame G_{t-1}, raw detections D
Ensure: filtered detections ready for queue-zone processing

$G_t \leftarrow$ grey-scale version of F_t
$\mathcal{D}_{\text{ok}} \leftarrow \emptyset$
for all $d \in D$ with box (x_1, y_1, x_2, y_2) and confidence conf **do**
    $A \leftarrow (x_2 - x_1)(y_2 - y_1)$; $f \leftarrow A/(W_f H_f)$; $r \leftarrow (y_2 - y_1)/(x_2 - x_1)$
    if $A < A_{\min}$ **or** $f > f_{\max}$ **or** $r < r_{\min}$ **then**
        continue
    end if
    compute motion energy M from G_t and G_{t-1} using Equation 3.5
    if $M < M_{\min}$ **and** conf $< c_{\text{byp}}$ **then**
        continue
    end if
    append d to $\mathcal{D}_{\text{ok}}$
end for
apply ByteTrack/NMS to $\mathcal{D}_{\text{ok}}$
return filtered tracked detections

Table 3.1: Computer Vision Layer parameters.

Symbol	Value	Description
$A_{\min}$	800 px^2	Minimum bounding-box area (Gate 1)
$f_{\max}$	0.85	Maximum frame coverage (Gate 2)
$r_{\min}$	0.60	Minimum portrait aspect ratio (Gate 3)
$M_{\min}$	8 px	Minimum motion energy (Gate 4)
c_{byp}	0.70	Static-detection confidence bypass (Gate 4)
α_{s}	0.45	Bounding-box EMA smoothing factor

3.2.2.2 Bounding-box smoothing

Raw YOLO bounding boxes jitter from frame to frame even when the underlying person is moving smoothly. Each of the four box coordinates is therefore smoothed with an exponential moving average:

$$\hat{b}_t \;=\; \alpha_{\text{s}}\, b_t \;+\; (1-\alpha_{\text{s}})\, \hat{b}_{t-1}, \tag{3.6}$$

applied independently to x_1, y_1, x_2, and y_2. The choice $\alpha_{\text{s}} = 0.45$ is a practical midpoint for walking-speed motion at 12–15 fps: a higher α_{s} reacts faster but appears jittery, a lower value is smoother but lags behind. The smoothed boxes are the ones rendered into the annotated snapshot that the staff dashboard displays.

3.2.3 Queue Zone Membership and Candidate Confirmation

A configurable rectangular region of interest, the queue zone, is defined by four pixel coordinates $(z_{x_1}, z_{y_1}, z_{x_2}, z_{y_2})$ on the camera frame. A detected person is considered *inside* the zone when the centroid of their bounding box, $(c_x, c_y) = \big((x_1+x_2)/2,\ (y_1+y_2)/2\big)$, falls within the rectangle:

$$\text{inside} \iff z_{x_1} \le c_x \le z_{x_2} \;\wedge\; z_{y_1} \le c_y \le z_{y_2}. \tag{3.7}$$

The centroid is preferred over any single edge because the centroid does not flicker into the zone when only an arm or shoulder crosses the boundary.

A person who has just entered the queue zone is not assigned a queue number immediately. Each new track identifier is buffered in a candidate dictionary C and must satisfy two conditions before a number is issued: persistence and motion. Persistence requires that the same tid remain inside the zone for at least N_{conf} consecutive frames. Motion is assessed from the standard deviations of the buffered centroid sequence $\{(c_{x,i}, c_{y,i})\}_{i=1}^{N_{\text{conf}}}$:

$$\sigma_x \;=\; \text{std}(c_{x,1}, \ldots, c_{x,N_{\text{conf}}}), \qquad \sigma_y \;=\; \text{std}(c_{y,1}, \ldots, c_{y,N_{\text{conf}}}), \tag{3.8}$$

$$\text{motion ok} \iff \sigma_x + \sigma_y > \theta_{\text{m}} \;\vee\; \overline{\text{conf}} \ge c_{\text{byp}}. \tag{3.9}$$

Algorithm 2 formalises the confirmation procedure used after the zone and motion tests above.

Algorithm 2 Candidate confirmation

Require: detection d with track ID tid and confidence conf, candidate buffer C, threshold N_{conf}
Ensure: issue queue number, continue accumulating, or reject as static

C[tid].count $\leftarrow C$[tid].count $+ 1$
append (c_x, c_y) to C[tid].centers
append conf to C[tid].confs
if C[tid].count $< N_{\text{conf}}$ **then**
    return continue accumulating
end if
compute σ_x, σ_y over C[tid].centers
$\overline{\text{conf}} \leftarrow \text{mean}(C[\text{tid}].\text{confs})$
if $\sigma_x + \sigma_y > \theta_{\text{m}}$ **or** $\overline{\text{conf}} \geq c_{\text{byp}}$ **then**
    assign next queue number to tid
else
    delete C[tid] ▷ static object rejected
end if

At $N_{\text{conf}} = 20$ frames and a typical frame rate of 12–15 fps, a person must remain in the zone for approximately 1.3–1.7 s before receiving a queue number. The motion threshold $\theta_{\text{m}} = 1.5$ px rejects static objects (signs, chairs, items left on the floor) that YOLO occasionally mis-classifies as persons; the confidence bypass at 0.70 admits a momentarily stationary person whose detection is otherwise unambiguous.

3.2.4 Track-Identifier Remapping

ByteTrack occasionally assigns a new tid to a person who briefly left and re-entered the camera frame. Before any unfamiliar identifier is forwarded to the queue tracker as a candidate, `remap_track_ids` attempts to recover the original identifier by comparing the new detection's bounding box to those of currently active queue persons. Two signals are used: IoU (Equation 3.1) and centroid Euclidean distance,

$$d(A, B) \;=\; \sqrt{(c_x^A - c_x^B)^2 + (c_y^A - c_y^B)^2}. \tag{3.10}$$

A remap score is computed with IoU prioritised over distance:

$$s \;=\; \begin{cases} \text{IoU}(A, B) + 1.0 & \text{if IoU}(A, B) \geq \tau_{\text{iou}}^{\text{rm}}, \\ 1 - d(A, B)/d_{\max} & \text{else if } d(A, B) \leq d_{\max}, \\ (\text{no match}) & \text{otherwise.} \end{cases} \tag{3.11}$$

The additive constant 1.0 in the IoU branch guarantees that any genuine overlap always outranks a distance-only match. The candidate with the highest score above either threshold inherits the known identifier. With $\tau_{\text{iou}}^{\text{rm}} = 0.10$ and $d_{\max} = 180$ px, remapping rescues brief tracker hiccups without false matches across the zone.

Algorithm 3 Track-identifier remapping

Require: incoming detection d with new track ID tid_{new}, active queue $\mathcal{Q}$
Ensure: original track ID if recoverable, otherwise tid_{new}

```
bestID ← tid_new; bestScore ← −∞
for all p ∈ Q with active or recently visible status do
    u ← IoU(d.bbox, p.bbox)
    δ ← d(d.bbox, p.bbox)
    if u ≥ τ_iou^rm then
        s ← u + 1.0
    else if δ ≤ d_max then
        s ← 1 − δ/d_max
    else
        continue
    end if
    if s > bestScore then
        bestScore ← s; bestID ← p.tid
    end if
end for
return bestID
```

3.2.5 Duplicate Suppression

A second class of tracker artefact, distinct from a re-entered person, is the *ghost track*: two different tid values assigned to the same physical person within the same frame. Ghost tracks must be merged in the active queue, retaining the lower queue number. The merge condition combines bounding-box overlap and centroid proximity:

$$\text{dup}(p_i, p_j) \iff \text{IoU}(p_i, p_j) > \tau_{\text{iou}}^{\text{dd}} \ \vee \ d(p_i, p_j) < f_{\text{dd}} \cdot \overline{D}(p_i, p_j), \tag{3.12}$$

where $\overline{D}(p_i, p_j)$ is the mean diagonal of the two bounding boxes and f_{dd} is the centre-fraction threshold. Two safeguards prevent over-aggressive merging. First, an immunity counter is set to $I_0 = 60$ frames whenever a queue entry is restored via re-identification (§3.2.6); while the counter is positive, the entry is exempt from deduplication. Second, even when Equation (3.12) is satisfied, the merge is rejected if the appearance similarity (Equation 3.16) between the two entries falls below $\tau_{\text{bl}} = 0.55$, on the grounds that two visually distinct persons should not be collapsed regardless of their geometric proximity. Parameter values are listed in Table 3.2.

Table 3.2: Duplicate-suppression parameters.

Symbol	Value	Description
$\tau_{\text{iou}}^{\text{dd}}$	0.40	IoU threshold for ghost-track merge
f_{dd}	0.15	Centre-distance fraction of mean diagonal
I_0	60	Re-entry immunity (frames)
τ_{bl}	0.55	Appearance guard against false merge

Algorithm 4 formalises the per-frame deduplication pass.

Algorithm 4 Per-frame duplicate suppression

Require: active queue $\mathcal{Q}$, immunity table I
for all ordered pairs $(p_i, p_j) \in \mathcal{Q} \times \mathcal{Q}$ with $i < j$ **do**
  if $I[p_i] > 0$ **or** $I[p_j] > 0$ **then**
    continue ▷ immunity active
  end if
  if appearance signatures exist **and** $\mathrm{sim}(p_i, p_j) < \tau_{\mathrm{bl}}$ **then**
    continue ▷ visually distinct
  end if
  if $\mathrm{dup}(p_i, p_j)$ holds (Equation 3.12) **then**
    drop the higher queue number, keep the lower
  end if
end for
decrement every positive entry of I by one

3.2.6 Appearance-Based Re-identification

Re-identification recovers the original queue number when a person leaves the camera frame for longer than the tracker can bridge, then returns. The procedure has four parts: an appearance signature (§3.2.6.1), a comparison metric (§3.2.6.2), a hybrid decision rule (§3.2.6.3), and two guards against false matches (§3.2.6.4).

3.2.6.1 Appearance signature

For each queued person, the bounding-box crop is divided horizontally at its midpoint and a 16×16 joint Hue–Saturation histogram is computed for each half in the HSV colour space:

$$H(b_h, b_s) \;=\; \big|\{p : \mathrm{hue}(p) \in B_h \wedge \mathrm{sat}(p) \in B_s\}\big|, \tag{3.13}$$

with $b_h \in \{0, \ldots, 15\}$ partitioning the OpenCV hue range $[0, 180)$ and $b_s \in \{0, \ldots, 15\}$ partitioning the saturation range $[0, 256)$. Each half-histogram is normalised so its bin values sum to one. The full appearance signature $\mathrm{sig} \in \mathbb{R}^{512}$ is the concatenation of the upper and lower half-histograms. HSV is preferred over RGB because separating chroma from luminance makes the signature more robust to indoor–outdoor lighting variation. The signature is updated every 30 s of wait time using an exponential moving average:

$$\mathrm{sig}_t \;=\; 0.6\,\mathrm{sig}_{t-1} + 0.4\,\mathrm{sig}_{\mathrm{new}}. \tag{3.14}$$

3.2.6.2 Comparison metric

Two signatures are compared using the Pearson histogram correlation applied to each body half:

$$r(H_1, H_2) \;=\; \frac{\sum_i (H_{1,i} - \bar{H}_1)(H_{2,i} - \bar{H}_2)}{\sqrt{\sum_i (H_{1,i} - \bar{H}_1)^2}\,\sqrt{\sum_i (H_{2,i} - \bar{H}_2)^2}}, \tag{3.15}$$

and combined with a fixed body-half weighting that favours the upper body, which carries more person-distinctive colour information (shirts, jackets) than the lower body:

$$\mathrm{sim}(\mathrm{sig}_1, \mathrm{sig}_2) \;=\; 0.6\, r(H_1^{\mathrm{upper}}, H_2^{\mathrm{upper}}) + 0.4\, r(H_1^{\mathrm{lower}}, H_2^{\mathrm{lower}}). \tag{3.16}$$

The metric ranges over $[-1, 1]$. Three thresholds are applied across the system: $\tau_{\mathrm{tb}} = 0.30$ for the re-identification tiebreak (§3.2.6.3), $\tau_{\mathrm{bl}} = 0.55$ for the done-blacklist match (§3.2.6.4), and $\tau_{\mathrm{twin}} = 0.85$ for the twin/lookalike guard (§3.2.6.4).

3.2.6.3 Hybrid decision rule

When a new track identifier appears that did not match any active queue person under remapping (§3.2.4), the re-identifier considers all missing-person records eligible within the $T_{\mathrm{rec}} = 120$ s recency window. The decision rule has two branches.

In the single-eligible-missing-person branch, the appearance comparison is applied first: if $\mathrm{sim}(\mathrm{sig}, \mathrm{sig}_{\mathrm{new}}) \geq \tau_{\mathrm{tb}}$, the missing queue number is restored. If appearance is inconclusive, a spatial fallback applies: with normalised spatial score

$$s_{\mathrm{sp}}(b_1, b_2) \;=\; \frac{d(b_1, b_2)}{\frac{1}{2}\big(\mathrm{diag}(b_1) + \mathrm{diag}(b_2)\big)}, \tag{3.17}$$

the queue number is restored if $s_{\mathrm{sp}} < s_{\mathrm{fb}}^{\mathrm{app}}$, with $s_{\mathrm{fb}}^{\mathrm{app}} = 0.20$ (a centroid offset of roughly one-fifth of an average bounding-box diagonal). If neither condition holds, a new queue number is issued.

In the multi-eligible-missing-person branch, the spatial fallback is deliberately omitted: a person who returns to a different queue position would otherwise be matched to the entry that happens to sit nearest in pixel space, which is precisely the wrong record. Instead, the appearance scores of all eligible missing persons are ranked, and restoration is committed only when the best score $s^{(1)}$ exceeds τ_{tb} *and* exceeds the second-best score $s^{(2)}$ by at least $\Delta_{\mathrm{mm}} = 0.10$:

$$\text{restore} \iff s^{(1)} \geq \tau_{\mathrm{tb}} \;\wedge\; s^{(1)} - s^{(2)} \geq \Delta_{\mathrm{mm}}. \tag{3.18}$$

Otherwise the system declines to guess and issues a new number. The branch logic above is applied only after the guard checks below have not blocked the detection.

3.2.6.4 Twin guard and done blacklist

Two guards run before any re-identification commit. The twin guard fires when the new detection is nearly identical in appearance to a person already visible in the active queue:

$$\exists\, p \in \mathcal{Q} \;:\; \mathrm{sim}(p.\mathrm{sig}, \mathrm{sig}_{\mathrm{new}}) \geq \tau_{\mathrm{twin}}. \tag{3.19}$$

When the condition holds, restoration is blocked and a fresh queue number is issued; the rationale is that two persons with near-identical clothing should not be merged into a single queue entry. The done blacklist fires when the new detection's appearance matches that of a person already served in the same session:

$$\exists\, b \in \mathcal{B} \;:\; \mathrm{sim}(b, \mathrm{sig}_{\mathrm{new}}) \geq \tau_{\mathrm{bl}}, \tag{3.20}$$

where $\mathcal{B}$ is the set of appearance signatures of served persons retained for the duration of the session. When the condition holds, no new queue entry is created. The thresholds $\tau_{\text{twin}} = 0.85$ and $\tau_{\text{bl}} = 0.55$ reflect the asymmetric tolerance these guards require: the twin guard must be strict (only near-identical appearances trigger it) so that ordinary returning persons are not mis-classified as twins, whereas the done blacklist must be more permissive so that a recently served person whose appearance has shifted slightly (due to lighting or posture) is still recognised.

Algorithm 5 gives the complete re-identification order: guard checks first, single-missing or multi-missing recovery second. The calling queue service treats BLOCKED as no ticket and NONE as a new entrant that may receive a fresh queue number.

Algorithm 5 Hybrid re-identification (appearance-first with spatial fallback)

Require: new detection signature sig_{new}, box b_{new}, active queue $\mathcal{Q}$, served blacklist $\mathcal{B}$, eligible missing set $\mathcal{M}$
Ensure: restored queue person, NONE, or BLOCKED

if $\exists p \in \mathcal{Q} : \text{sim}(p.\text{sig}, \text{sig}_{\text{new}}) \geq \tau_{\text{twin}}$ **then**
  return NONE ▷ lookalike guard; issue a fresh number
end if
if $\exists b \in \mathcal{B} : \text{sim}(b, \text{sig}_{\text{new}}) \geq \tau_{\text{bl}}$ **then**
  return BLOCKED ▷ already served in this session
end if
if $|\mathcal{M}| = 1$ **then** ▷ single-missing branch
  $p \leftarrow$ the unique element of $\mathcal{M}$
  $s \leftarrow \text{sim}(p.\text{sig}, \text{sig}_{\text{new}})$
  if $s \geq \tau_{\text{tb}}$ **then return** p ▷ appearance match
  end if
  if $s_{\text{sp}}(p.\text{bbox}, b_{\text{new}}) < s_{\text{fb}}^{\text{app}}$ **then return** p ▷ spatial fallback
  end if
  return NONE
else ▷ multi-missing branch
  rank $\mathcal{M}$ by $\text{sim}(\cdot, \text{sig}_{\text{new}})$
  let $s^{(1)}, s^{(2)}$ be the top scores; let $p^{(1)}$ be the best match
  if $s^{(1)} \geq \tau_{\text{tb}}$ **and** $s^{(1)} - s^{(2)} \geq \Delta_{\text{mm}}$ **then**
    return $p^{(1)}$
  else
    return NONE ▷ ambiguous
  end if
end if

3.2.7 No-Show Detection

When the person at position 1 in the active queue becomes missing, a countdown is started. If the person does not return before the no-show window $T_{\text{ns}} = 300$ s elapses, the entry is automatically marked as a no-show and bumped from the queue. Let $\tau_{\text{start}}(q)$ denote the moment the countdown for queue entry q begins. The elapsed time is

$$e(q) \;=\; t_{\text{now}} - \tau_{\text{start}}(q), \tag{3.21}$$

and the no-show condition is

$$\text{no\_show}(q) \iff e(q) \geq T_{\text{ns}} \ \wedge \ \text{position}(q) = 1 \ \wedge \ \text{status}(q) = \texttt{missing}. \tag{3.22}$$

The countdown resets to zero if the person returns before expiry. The dashboard displays the remaining time $\max(0,\, T_{\text{ns}} - e(q))$ and tags the alert as `critical` when the remaining time falls below 15 s and `warning` otherwise. Only the position-1 entry is monitored because only that entry is being called; persons further back in the line may legitimately be waiting elsewhere within the service area.

Algorithm 6 No-show countdown and bumping

Require: active queue $\mathcal{Q}$, countdown table S, current time t_{now}
for all $q \in \mathcal{Q}$ **do**
  if q.position $= 1$ **and** q.status $=$ `missing` **then**
    if $q \notin S$ **then**
      $S[q] \leftarrow t_{\text{now}}$
    else if $t_{\text{now}} - S[q] \geq T_{\text{ns}}$ **then**
      mark q as `no_show`
      move q from active queue to completed/no-show records
      register q's appearance in the done blacklist
      delete $S[q]$
    end if
  else
    delete $S[q]$ if present
  end if
end for

3.2.8 Counter Assignment

After every frame, the active queue $\mathcal{Q}$ is sorted by queue number in ascending order. Each person receives an integer position-in-line and, if a counter is available, is assigned to one. Counter assignment is *sticky*: once a counter is allocated to a queue entry, the allocation persists until that entry is marked done or no-show, regardless of changes to the line behind it. Let $\text{rank}(p)$ denote the 1-based position of p.queue_number in the sorted active queue and let c be the number of counters configured for the service office. The assignment rule is

$$\text{counter}(p) \ = \ \begin{cases} \text{rank}(p) & \text{if } \text{rank}(p) \leq c \text{ and } p \text{ unassigned}, \\ \text{previous allocation} & \text{if } p \text{ already assigned}, \\ (\text{none}) & \text{otherwise}. \end{cases} \tag{3.23}$$

The sticky rule is essential for user experience: without it, every time the position-1 person is served, the remaining queue would visibly shift one counter to the left, confusing both the served-next student and the staff member at the affected counter. Algorithm 7 formalises the per-frame procedure.

Algorithm 7 Sticky counter assignment

Require: active queue $\mathcal{Q}$, number of counters c
sort $\mathcal{Q}$ by queue number ascending; call this L
for $i = 1$ to $|L|$ **do**
    $L[i]$.position $\leftarrow i$
end for
occupied $\leftarrow \{p.\text{counter} : p \in L \text{ with } p.\text{counter} \neq \text{none}\}$
free $\leftarrow \{1, \ldots, c\} \setminus$ occupied, sorted ascending
for all $p \in L$ with p.counter = none **do**
    if free non-empty **then**
        p.counter $\leftarrow$ pop smallest element of free
    end if
end for

3.2.9 Wait-Time Prediction

The Prediction Service consumes a rolling buffer of up to sixty crowd-count samples and updates the wait-time estimates every frame. It first computes the current wait estimate with an M/M/c approximation, then estimates the arrival rate, computes a local trend slope, and uses those values for the short-, medium-, and long-horizon forecasts. Section 3.2.10 (§3.2.10) describes the dynamic service-time measurement that supplies μ to the M/M/c baseline.

3.2.9.1 M/M/c baseline (current wait)

With arrival rate λ, service rate μ, and c active counters, the system utilisation $\rho = \lambda/(c\mu)$ is given by Equation 1.1. When the system is stable ($\rho < 1$), the current wait estimate combines Little's Law (Equation 1.2) with the expected service duration:

$$W_{\text{current}} = \begin{cases} \dfrac{Q}{\max(\lambda,\, 0.1)} + \dfrac{1}{\mu} & \text{if } \rho < 1, \\ \dfrac{Q}{\mu\, c} & \text{if } \rho \geq 1. \end{cases} \tag{3.24}$$

The saturated-system branch is a deterministic conservative estimate used when arrivals exceed capacity, in which case the classical M/M/c steady-state formulas no longer apply.

3.2.9.2 Arrival rate

The arrival rate over the recent history is estimated from the difference of queue counts n samples apart, divided by the elapsed time between those samples in minutes:

$$\lambda = \frac{Q_t - Q_{t-n}}{\Delta t}, \qquad n = \min(30,\, |\text{history}|). \tag{3.25}$$

Negative differences (departures exceeding arrivals) are clamped to zero, and the value is recomputed every frame.

3.2.9.3 Trend slope

The short-horizon forecast requires a local rate of change in queue count. A simple ordinary-least-squares regression is fit to the most recent $W_{\text{tr}} = 20$ samples after de-meaning the time indices, yielding a no-intercept slope

$$\beta \;=\; \frac{\sum_i x_i\, y_i}{\sum_i x_i^2}, \qquad x_i \;=\; i - \bar{x}, \tag{3.26}$$

where y_i is the queue count at sample i. The slope is expressed in persons per frame and is converted to persons per second in the next sub-model via an estimated frame rate.

3.2.9.4 Short-term forecast (≈ 5 min)

The projected queue count $T_{\text{sh}} = 600$ s ahead is obtained by linear extrapolation along the OLS slope:

$$\hat{Q}_{\text{short}} \;=\; \max\bigl(0,\, Q + \beta \cdot \text{fps} \cdot T_{\text{sh}}\bigr). \tag{3.27}$$

The projected wait time is then computed from $\hat{Q}_{\text{short}}$ using the same M/M/c-style conversion as Equation (3.24). Linear extrapolation is appropriate at this horizon because queue level rarely changes regime within five minutes during normal service operation.

3.2.9.5 Medium-term forecast (≈ 15 min)

For the medium horizon, Holt's double-exponential smoothing [18] is applied to the crowd-count buffer. The level L_t and the trend T_t are updated jointly each frame:

$$L_t \;=\; \alpha_{\text{H}}\, y_t \;+\; (1 - \alpha_{\text{H}})\,(L_{t-1} + T_{t-1}), \tag{3.28}$$

$$T_t \;=\; \beta_{\text{H}}\,(L_t - L_{t-1}) \;+\; (1 - \beta_{\text{H}})\, T_{t-1}. \tag{3.29}$$

The forecast h steps ahead is then damped by a factor $\varphi < 1$ raised to the projection step, preventing the trend component from over-projecting during transient spikes:

$$\hat{Q}_{t+h} \;=\; L_t \;+\; T_t \sum_{i=1}^{h} \varphi^i. \tag{3.30}$$

The deployed values are $\alpha_{\text{H}} = 0.4$, $\beta_{\text{H}} = 0.2$, $\varphi = 0.85$, and $h = 20$, yielding $\sum_{i=1}^{20} 0.85^i \approx 5.67$. The projected wait time is again obtained by applying the M/M/c-style conversion of Equation (3.24) to $\hat{Q}_{t+h}$.

3.2.9.6 Long-term forecast (≈ 30 min)

For the long horizon, linear or trend-only methods are unreliable. QueueFlow instead blends a growth-ratio projection with mean reversion to a recent anchor. The crowd-count history is split into thirds, with Q_{early} and Q_{recent} denoting the means of the earliest and latest thirds respectively. The growth ratio is

$$g \;=\; \frac{Q_{\text{recent}}}{\max(Q_{\text{early}},\,1)}, \tag{3.31}$$

and the projected queue at the long horizon is

$$\hat{Q}_{\text{long}} \;=\; \max\Big(0,\; Q_{\text{recent}} \cdot \big(0.7\,g + 0.3\big)\Big), \tag{3.32}$$

placing 70% of the weight on the observed growth trend and 30% on mean reversion to the recent level. The projected wait time is then derived as in Equation (3.24) and capped at sixty minutes; long-horizon forecasts are inherently uncertain and the cap prevents a single high-growth window from producing a meaningless multi-hour estimate on the dashboard.

Algorithm 8 Multi-horizon wait-time prediction

Require: count history H, current queue length Q, active counters c, service rate μ
Ensure: current, short-, medium-, and long-horizon wait estimates
1: append current Q and timestamp to H
2: compute λ from recent count differences using Equation 3.25
3: $\rho \leftarrow \lambda/(c\mu)$
4: $W_{\text{current}} \leftarrow$ Equation 3.24
5: compute OLS slope β on the latest W_{tr} samples using Equation 3.26
6: $\hat{Q}_{\text{short}} \leftarrow \max(0, Q + \beta \cdot \text{fps} \cdot T_{\text{sh}})$
7: $W_{\text{short}} \leftarrow$ convert $\hat{Q}_{\text{short}}$ to wait time using Equation 3.24
8: $(L_t, T_t) \leftarrow$ Holt update over H using Equations 3.28–3.29
9: $\hat{Q}_{t+h} \leftarrow L_t + T_t \sum_{i=1}^{h} \varphi^i$
10: $W_{\text{medium}} \leftarrow$ convert $\hat{Q}_{t+h}$ to wait time using Equation 3.24
11: split H into thirds and compute Q_{early} and Q_{recent}
12: $g \leftarrow Q_{\text{recent}} / \max(Q_{\text{early}}, 1)$
13: $\hat{Q}_{\text{long}} \leftarrow \max(0, Q_{\text{recent}}(0.7g + 0.3))$
14: $W_{\text{long}} \leftarrow \min(W_{\text{max}}^{\text{long}}$, converted wait from $\hat{Q}_{\text{long}})$
15: **return** $(W_{\text{current}}, W_{\text{short}}, W_{\text{medium}}, W_{\text{long}}, \rho, \lambda)$

3.2.9.7 Hybrid AI wait-time predictor

The deployed AI model does not replace the analytical routines above. Instead, QueueFlow treats their outputs as interpretable engineered features for a supervised Gradient Boosting regressor. For each training or live prediction row, the system computes queue-state features, recent-demand features, academic-period indicators, and the analytical prediction features W_{current}, W_{short}, W_{medium}, and W_{long} from Algorithm 8. These features are passed to a `HistGradientBoostingRegressor`, which learns nonlinear corrections from historical queue behavior.

During live operation, the trained model is not retrained. The backend calibrates the AI output with runtime evidence: current service speed, active counters, yesterday's same-hour average wait, and recent prediction error. This hybrid design keeps the prediction grounded in the queueing and forecasting framework while allowing the model to learn from historical data.

Algorithm 9 Hybrid AI wait-time prediction

Require: current queue state, academic calendar, count history H, active counters c, trained Gradient Boosting model G
Ensure: calibrated student-facing wait-time estimate
compute queue-state features: queue number, people ahead, and daily queue size
compute recent-demand features: arrivals in the last 5 and 15 minutes
compute academic-period indicators from the academic calendar
compute analytical prediction features using Algorithm 8
concatenate all features into vector x
$W_{\text{AI}} \leftarrow G(x)$
compute recent prediction-error correction e_{recent}
$W_{\text{AI,corr}} \leftarrow \max(0, W_{\text{AI}} + e_{\text{recent}})$
compute runtime baseline W_{base} from current service speed, active counters, and yesterday same-hour wait
$\gamma \leftarrow \min(\text{completed\_today}/10, 1)$ ▷ confidence in current-day service pattern
blend W_{base} toward current-day service behavior using γ
$W_{\text{final}} \leftarrow 0.6W_{\text{AI,corr}} + 0.4W_{\text{base}}$
return bounded W_{final}

3.2.10 Dynamic Service-Time Measurement

The service rate $\mu = 1/\overline{\text{svc}}$ used in Equation (3.24) is not a static parameter. A background thread re-measures the average service time every $T_{\text{svc}} = 300$ s from the MySQL transaction log, ensuring that the prediction component adapts to the actual pace of service rather than to a single pre-deployment calibration.

For a sequence of completed-transaction timestamps $t_1, t_2, \ldots, t_N$, inter-departure gaps in minutes are

$$g_i \;=\; \frac{t_i - t_{i-1}}{60}, \qquad i = 2, \ldots, N, \tag{3.33}$$

and gaps outside the range [0.1, 15] min are discarded as either database-write noise (very short) or cashier idle periods (very long). The mean of the retained gaps is multiplied by c to correct for the fact that, in a parallel-server system, the inter-departure interval observed at the aggregate output is approximately $\overline{\text{svc}}/c$:

$$\widehat{\text{svc}} \;=\; \bar{g} \cdot c. \tag{3.34}$$

The new estimate is then blended with the previously stored value so that a single unusually fast or slow batch of transactions does not jolt the prediction:

$$\widehat{\text{svc}}_{\text{new}} \;=\; 0.7\,\widehat{\text{svc}} + 0.3\,\widehat{\text{svc}}_{\text{prev}}. \tag{3.35}$$

The pre-deployment calibration described in Section 2.4 (§2.4, Phase 1) supplies the initial seed value for $\widehat{\text{svc}}_{\text{prev}}$ before any transactions have been logged.

Algorithm 10 Dynamic service-time measurement

Require: served transaction timestamps $t_1, \ldots, t_N$, active counters c, previous service time $\widehat{\text{svc}}_{\text{prev}}$
Ensure: updated service time $\widehat{\text{svc}}_{\text{new}}$ and service rate μ

$G \leftarrow \emptyset$
for $i = 2$ to N **do**
  $g_i \leftarrow (t_i - t_{i-1})/60$
  if $0.1 \le g_i \le 15$ **then**
    append g_i to G
  end if
end for
if G has too few samples **then**
  return previous $\widehat{\text{svc}}_{\text{prev}}$ and $\mu = 1/\widehat{\text{svc}}_{\text{prev}}$
end if
$\widehat{\text{svc}} \leftarrow \text{mean}(G) \cdot c$
$\widehat{\text{svc}}_{\text{new}} \leftarrow 0.7\widehat{\text{svc}} + 0.3\widehat{\text{svc}}_{\text{prev}}$
$\mu \leftarrow 1/\widehat{\text{svc}}_{\text{new}}$
return $(\widehat{\text{svc}}_{\text{new}}, \mu)$

3.2.11 Ticket Generation

When a queue number is confirmed, the Ticket Service generates a printer-agnostic ticket payload comprising four fields: the queue number, a short alphanumeric code for manual entry, a QR-encoded status-page URL, and a JSON Web Token (JWT) that authenticates subsequent status requests from the mobile client. During evaluation, the payload is rendered as a PDF saved to a local `Tickets/` directory. In office operation, the same queue record is sent to the thermal printer and issued as a paper ticket. The PDF or printed ticket is automatically deleted or voided when the entry is marked done or no-show.

The JWT is constructed with the HS256 algorithm. The header and payload are URL-safe base64-encoded JSON objects, concatenated by a period, and signed with a server-side secret using HMAC-SHA256:

$$\text{token} \;=\; h \cdot . \cdot p \cdot . \cdot \text{base64url}\big(\text{HMAC-SHA256}(h \cdot . \cdot p,\; K)\big), \tag{3.36}$$

where $h = \text{base64url}(\{\texttt{alg} : \texttt{"HS256"}, \texttt{typ} : \texttt{"JWT"}\})$, $p = \text{base64url}(\{\texttt{sub}, \texttt{iat}, \texttt{exp}, \texttt{jti}\})$, and K is the secret. The payload includes the queue number (`sub`), the issue time (`iat`), an expiry four hours later (`exp`), and a 32-character hexadecimal nonce (`jti`) that makes every issued token unique and prevents replay of an intercepted token.

The short code is drawn uniformly at random from the 32-character alphabet $\{A, \ldots, Z\} \cup \{2, \ldots, 9\} \setminus \{O, I\}$, excluding the visually ambiguous characters 0, O, 1, and I to eliminate manual-entry errors. Eight characters are drawn and displayed as `XXXX-XXXX`, providing entropy of $\log_2(32^8) \approx 40$ bits, sufficient to render brute-force guessing infeasible within the four-hour ticket lifetime.

Algorithm 11 Printer-agnostic ticket generation

Require: confirmed queue number q, status-page base URL U, signing secret K
Ensure: persisted ticket payload and rendered ticket output
draw eight random characters from the unambiguous 32-character alphabet
format the short code as `XXXX-XXXX`
$jti \leftarrow$ random 32-character hexadecimal nonce
iat $\leftarrow$ current timestamp; exp $\leftarrow$ iat + 4 hours
create JWT header h and payload $p = \{\text{sub} : q, \text{iat}, \text{exp}, \text{jti}\}$
sign $h.p$ with HMAC-SHA256 and secret K to form token
create QR URL $U(q, \text{token})$
persist queue number, short code, QR URL, token, expiry, and status
if prototype output is configured **then**
    render the payload as a PDF ticket
else
    send the same payload to the thermal-printer driver
end if
return ticket payload

3.2.12 Public Display Board with TTS Announcements

A third presentation surface, alongside the staff dashboard and the student mobile client, is a public display board mounted in the service-office waiting area. The board renders the active queue and announces newly assigned counter calls using the browser's built-in Web Speech API. All speech synthesis runs client-side; no audio is transmitted to any third-party service and no backend audio endpoint is required, which is consistent with the privacy posture established in Section 2.6 (§2.6).

A deduplication set indexed by the pair (queue_number, counter_number) prevents the same counter assignment from being announced more than once even when the polling endpoint returns the same active queue across consecutive intervals. Announcements are spoken sequentially with an 800 ms gap between utterances to avoid speech overlap when several counter assignments are issued simultaneously.

Algorithm 12 Public-display announcement deduplication

Require: latest counter assignments A, announced set E, speech queue P
for all $a \in A$ **do**
    $k \leftarrow (a.\text{queue\_number}, a.\text{counter\_number})$
    if $k \notin E$ **then**
        add k to E
        append announcement for a to P
    end if
end for
while P is non-empty **and** speech engine is idle **do**
    speak the next announcement in P
    wait 800 ms before speaking the next item
end while

3.2.13 Parameter Summary

Table 3.3 consolidates every tunable parameter referenced in this section. Values listed are those used in the deployed prototype.

Table 3.3: Consolidated parameter values for the deployed QueueFlow prototype.

Subsystem	Symbol	Value	Description
CV Layer	$A_{\min}$	800 px^2	Min. box area
	$f_{\max}$	0.85	Max. frame fraction
	$r_{\min}$	0.60	Min. portrait aspect
	$M_{\min}$	8 px	Min. motion energy
	c_{byp}	0.70	Static-conf bypass
	α_{s}	0.45	Bbox EMA factor
Candidate conf.	N_{conf}	20 frames	Confirmation window
	θ_{m}	1.5 px	Motion-gate σ
Remap	$\tau_{\text{iou}}^{\text{rm}}$	0.10	Remap IoU threshold
	$d_{\max}$	180 px	Remap distance cap

Table 3.3 continued.

Subsystem	Symbol	Value	Description
Dedup	$\tau_{\text{iou}}^{\text{dd}}$	0.40	Dedup IoU threshold
	f_{dd}	0.15	Centre-fraction gate
	I_0	60 frames	Re-entry immunity
Re-ID	τ_{tb}	0.30	Appearance tiebreak
	τ_{bl}	0.55	Done blacklist / appearance guard
	τ_{twin}	0.85	Twin guard
	$s_{\text{fb}}^{\text{app}}$	0.20	Spatial fallback (single)
	Δ_{mm}	0.10	Multi-missing score gap
	T_{rec}	120 s	Re-entry recency window
No-show	T_{ns}	300 s	No-show window
Prediction	α_{H}	0.4	Holt's level smoothing
	β_{H}	0.2	Holt's trend smoothing
	φ	0.85	Holt's damping factor
	h	20	Holt's projection steps
	W_{tr}	20 samples	Trend regression window
	T_{sh}	600 s	Short-term horizon
	$W_{\max}^{\text{long}}$	60 min	Long-term cap
Service time	T_{svc}	300 s	Re-measure interval
Ticket	N/A	4 h	JWT expiry
	N/A	40 bits	Short-code entropy
Display board	N/A	800 ms	TTS inter-utterance gap

3.3 Summary of Findings

3.4 Conclusions

3.5 Recommendations

A Synthesis Matrix of Related Literature

Table A.1 presents a synthesis matrix summarizing the reviewed literature, the methodology of each study, key findings, and the gaps identified that motivated the present work.

Table A.1: Synthesis Matrix of Related Literature

Author/s	Year	Topic/Focus	Findings	Gap/Limitation
Sindagi & Patel [9]	2018	CNN-based crowd counting and density estimation	Deep learning enables robust counting under occlusion and scale variation	No queue-number assignment or re-identification
Redmon & Farhadi [3]	2018	YOLOv3 real-time object detection	Single-pass network predicts bounding boxes and class probabilities	No queue-specific deduplication
Jocher et al. [5]	2023	YOLOv8 multi-scale anchor-free detection	Accuracy-speed tradeoff with persistent tracking support	Duplicate head/body boxes in close-range scenarios
Wang et al. [6]	2025	Lightweight YOLOv8n-derived person detector	Supports edge-device surveillance person detection with reduced overhead	Surveillance-focused; no queue ticketing or wait-time prediction
Jocher & Qiu [7]	2024	YOLO11 object detection software	Improves model efficiency and feature reuse in the Ultralytics family	Newer variant; not selected for stable prototype integration
Tian et al. [8]	2025	Attention-centric YOLOv12 detector	Introduces attention-oriented architecture for real-time detection	Benchmark-focused; added complexity for campus prototype hardware
Terven et al. [10]	2023	Comprehensive review of YOLO architectures	Documents innovations from YOLOv1 to YOLOv8 and YOLO-NAS	General review; no queue-specific application
Wojke et al. [12]	2017	DeepSORT online multi-object tracking	Appearance and motion association reduce identity switches during occlusion	No explicit queue-position semantics
Ye et al. [13]	2022	Deep learning for person re-identification	Categorizes feature learning, metric learning, and ranking approaches	No integration with queue management
Wang et al. [14]	2024	Person and vehicle re-identification survey	Reviews recent Re-ID advances and robustness challenges	Broader Re-ID scope; no campus queue workflow
Asperti et al. [15]	2025	Recent person re-identification techniques	Documents newer representation and robustness approaches	Deep Re-ID methods add overhead for a single-zone prototype
Padilla-Lopez et al. [11]	2015	Visual privacy protection methods	Categorizes anonymization, scrambling, and aggregation techniques	Not specific to queue or service-area monitoring
Concepcion et al. [1]	2026	Smart queuing and appointment system in a Philippine HEI	Web-based queue and appointment management can improve client services	User-initiated ticketing; no passive computer-vision arrival detection
Shortle et al. [17]	2018	Foundational queueing theory	M/M/c and Little's Law enable wait-time calculation	No integration with CV inputs
Holt [18]	2004	Exponentially weighted moving average forecasting	Trend and level smoothing capture short-term dynamics	No queue-specific application
Shmueli & Koppius [16]	2011	Predictive analytics as a research paradigm	Empirical prediction supports decision-making	Not applied to live queue systems
Hyndman & Athanasopoulos [19]	2021	Forecasting principles and practice	Treats exponential smoothing and multi-horizon forecasting	Not domain-specific to service queues

B QueueFlow Evaluation Questionnaire

FOR CONTENT VALIDATION – DRAFT

Naga College Foundation, Inc. – College of Computer Studies

Research Title: QueueFlow: Computer Vision Based Automated Queue Ticket Assignment and Real-Time Predictive System for Dynamic Campus Service Delivery

Researchers: Archie D. Balbin and Joshua Rovic P. Sanota

Pre-Screening

Are you 18 years of age or older?

☐ Yes ☐ No

Respondents who answer No are thanked and are not included in the study. No profile answers or survey responses are collected from them.

Section I: Respondent Profile

1. Respondent code assigned by researchers: _____
2. Respondent group: ☐ Service Staff ☐ Administrator / IT Personnel ☐ Student
3. Age range: ☐ 18–22 ☐ 23–30 ☐ 31–40 ☐ 41–50 ☐ 51 and above
4. Gender (optional): _____
5. For students only – Year level: ☐ 1st year ☐ 2nd year ☐ 3rd year ☐ 4th year ☐ 5th year or above
6. For students only – Program / Course: _____
7. For staff and administrators only – Length of service at NCF: ☐ Less than 1 year ☐ 1–3 years ☐ 4–7 years ☐ 8 years or more
8. During the evaluation period, how often did you interact with QueueFlow? ☐ Daily ☐ A few times per week ☐ Weekly ☐ Less than weekly

Instructions

For each statement below, choose the box that best matches your actual experience with QueueFlow during the evaluation period. The items ask about the parts of the

system used by your respondent group. When an item is outside your role, leave it blank or mark "Not Applicable" if that option is provided on the printed form.

Table B.1: Likert Scale Interpretation

Scale	**Response**	**Interpretation**
5	Strongly Agree	Strongly Agree / Excellent
4	Agree	Agree / Very Satisfactory
3	Neutral	Neutral / Satisfactory
2	Disagree	Disagree / Poor
1	Strongly Disagree	Strongly Disagree / Very Poor

Tag legend: [S] = Service Staff, [A] = Administrators / IT, [ST] = Students.

Table B.2: Dimension A: Usability

#	Item	Tag	5	4	3	2	1
A1	The QueueFlow dashboard is easy to navigate.	[S] [A]	☐	☐	☐	☐	☐
A2	The actions I need to perform, such as call next, mark done, and mark no-show, are easy to find on the dashboard.	[S]	☐	☐	☐	☐	☐
A3	I can complete my common tasks in QueueFlow without needing help.	[S] [A]	☐	☐	☐	☐	☐
A4	The QueueFlow mobile interface is easy to use on my phone.	[ST]	☐	☐	☐	☐	☐
A5	Scanning the QR code on my queue ticket is straightforward.	[ST]	☐	☐	☐	☐	☐
A6	Checking my queue status on the mobile interface is quick.	[ST]	☐	☐	☐	☐	☐

Table B.3: Dimension B: Detection Reliability

#	Item	Tag	5	4	3	2	1
B1	QueueFlow correctly assigns queue numbers to people who enter the queue area.	[S] [A] [ST]	☐	☐	☐	☐	☐
B2	QueueFlow rarely assigns duplicate queue numbers to the same person.	[S] [A]	☐	☐	☐	☐	☐
B3	When a person briefly leaves the queue area and returns, QueueFlow correctly recognizes them.	[S] [A]	☐	☐	☐	☐	☐
B4	QueueFlow correctly identifies when someone has left the queue without being served.	[S]	☐	☐	☐	☐	☐
B5	I trust the queue list shown on the QueueFlow dashboard.	[S] [A]	☐	☐	☐	☐	☐
B6	The queue number I received matched my actual position in the line.	[ST]	☐	☐	☐	☐	☐

Table B.4: Dimension C: Prediction Usefulness

#	Item	Tag	5	4	3	2	1
C1	The current wait-time estimate matches my actual experience.	[S] [A] [ST]	☐	☐	☐	☐	☐
C2	The 5-minute, 15-minute, and 30-minute wait-time predictions are useful for planning my work.	[S] [A]	☐	☐	☐	☐	☐
C3	The wait-time predictions help me decide when to come back if I cannot wait now.	[ST]	☐	☐	☐	☐	☐
C4	The prediction information is easy to understand.	[S] [A] [ST]	☐	☐	☐	☐	☐
C5	I find the prediction cards on the dashboard useful for managing the queue.	[S] [A]	☐	☐	☐	☐	☐
C6	The predictions are accurate enough to be useful to me.	[S] [A] [ST]	☐	☐	☐	☐	☐

Table B.5: Dimension D: Dashboard Clarity

#	Item	Tag	5	4	3	2	1
D1	The live camera snapshot on the dashboard is clear and easy to interpret.	[S] [A]	☐	☐	☐	☐	☐
D2	The queue list is easy to read at a glance.	[S] [A]	☐	☐	☐	☐	☐
D3	The analytics cards present information in a useful way.	[S] [A]	☐	☐	☐	☐	☐
D4	No-show alerts are clearly visible when they appear.	[S]	☐	☐	☐	☐	☐
D5	The counter controls are clearly labeled.	[S]	☐	☐	☐	☐	☐
D6	The dashboard layout makes it easy to find the information I need.	[S] [A]	☐	☐	☐	☐	☐

Table B.6: Dimension E: Overall Satisfaction

#	Item	Tag	5	4	3	2	1
E1	Overall, QueueFlow improves the queueing experience at the Cashiering Office.	[S] [A] [ST]	☐	☐	☐	☐	☐
E2	QueueFlow makes my work easier.	[S] [A]	☐	☐	☐	☐	☐
E3	QueueFlow makes my transactions less stressful.	[ST]	☐	☐	☐	☐	☐
E4	I would recommend continuing to use QueueFlow after this study.	[S] [A] [ST]	☐	☐	☐	☐	☐
E5	QueueFlow is a meaningful improvement over the previous manual queueing process.	[S] [A] [ST]	☐	☐	☐	☐	☐
E6	I am satisfied with QueueFlow overall.	[S] [A] [ST]	☐	☐	☐	☐	☐

Section VII: Open-Ended Questions

1. What did you like most about using QueueFlow?

2. What did you find most difficult or confusing when using QueueFlow?

3. If you could change one thing about QueueFlow, what would it be?

4. What Cashiering Office situations should QueueFlow handle better before it is considered for wider use?

5. Any other comments or suggestions?

End of Questionnaire. Thank you for your participation.

Questionnaire Notes

The instrument contains 30 Likert-scale statements: six each for Usability, Detection Reliability, Prediction Usefulness, Dashboard Clarity, and Overall Satisfaction. The open-ended questions are placed after the rating items so respondents can explain issues that may not appear in the numerical scores. As described in Sections 2.5 and 2.7, Cronbach's α is computed per dimension after the pilot run.

The tags identify which respondent group should answer each item. This allows the same evaluation form to support staff, administrator / IT, and student feedback while avoiding questions that do not fit a respondent's role. During scoring, dimension means are computed only from the items administered to that respondent group.

Before the questionnaire is used for the main evaluation, it is checked through expert content validation and pilot reliability testing. Items with weak item-total correlation are revised or removed before the final administration. If a Filipino version is needed, translation is done after the English version has passed validation, and the translated items are checked for equivalent meaning.

Bibliography

[1] A. T. V. Concepcion et al. "Smart Queuing and Appointment Management System for Client Services in Bulacan State University Bustos Campus". In: *International Journal of Multidisciplinary: Applied Business and Education Research* 7.1 (2026), pp. 5–14. DOI: 10.11594/ijmaber.07.01.02.

[2] Commission on Higher Education (CHED). *Policies and Guidelines on Outcomes-Based Education and Institutional Sustainability Assessment*. Tech. rep. Quezon City, Philippines: Commission on Higher Education, 2022.

[3] Joseph Redmon and Ali Farhadi. *YOLOv3: An Incremental Improvement*. 2018. arXiv: 1804.02767.

[4] Yifu Zhang et al. "ByteTrack: Multi-Object Tracking by Associating Every Detection Box". In: *Computer Vision – ECCV 2022*. Vol. 13682. Lecture Notes in Computer Science. Springer, 2022, pp. 1–21. DOI: 10.1007/978-3-031-20047-2_1. arXiv: 2110.06864.

[5] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. *Ultralytics YOLOv8*. https://github.com/ultralytics/ultralytics. 2023.

[6] Qiang Wang, Guo Feng, and Zhen Li. "A Lightweight Person Detector for Surveillance Footage Based on YOLOv8n". In: *Sensors* 25.2 (2025), p. 436. DOI: 10.3390/s25020436.

[7] Glenn Jocher and Jing Qiu. *Ultralytics YOLO11 (Version 11.0.0)*. https://github.com/ultralytics/ultralytics. 2024.

[8] Yunjie Tian, Qixiang Ye, and David Doermann. *YOLOv12: Attention-Centric Real-Time Object Detectors*. 2025. arXiv: 2502.12524.

[9] Vishwanath A. Sindagi and Vishal M. Patel. "A Survey of Recent Advances in CNN-based Single Image Crowd Counting and Density Estimation". In: *Pattern Recognition Letters* 107 (2018), pp. 3–16. DOI: 10.1016/j.patrec.2017.07.007.

[10] Juan Terven, Diana-Margarita Córdova-Esparza, and Julio-Alejandro Romero-González. "A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS". In: *Machine Learning and Knowledge Extraction* 5.4 (2023), pp. 1680–1716. DOI: 10.3390/make5040083.

[11] José Ramón Padilla-López, Alexandros Andre Chaaraoui, and Francisco Flórez-Revuelta. "Visual privacy protection methods: A survey". In: *Expert Systems with Applications* 42.9 (2015), pp. 4177–4195. DOI: 10.1016/j.eswa.2015.01.041.

[12] Nicolai Wojke, Alex Bewley, and Dietrich Paulus. "Simple Online and Realtime Tracking with a Deep Association Metric". In: *2017 IEEE International Conference on Image Processing (ICIP)*. Beijing, China: IEEE, 2017, pp. 3645–3649. DOI: 10.1109/ICIP.2017.8296962.

[13] Mang Ye et al. “Deep Learning for Person Re-Identification: A Survey and Outlook”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.6 (2022), pp. 2872–2893. DOI: 10.1109/TPAMI.2021.3054775.

[14] Z. Wang et al. “A Survey on Person and Vehicle Re-Identification”. In: *IET Computer Vision* 18.8 (2024), pp. 1235–1268. DOI: 10.1049/cvi2.12316.

[15] Andrea Asperti et al. “A Review of Recent Techniques for Person Re-Identification”. In: *Machine Vision and Applications* 36.1 (2025), pp. 1–29. DOI: 10.1007/s00138-024-01622-3.

[16] Galit Shmueli and Otto R. Koppius. “Predictive Analytics in Information Systems Research”. In: *MIS Quarterly* 35.3 (2011), pp. 553–572. DOI: 10.2307/23042796.

[17] John F. Shortle et al. *Fundamentals of Queueing Theory*. 5th ed. Hoboken, NJ: John Wiley & Sons, 2018. ISBN: 978-1-118-94352-6. DOI: 10.1002/9781119453765.

[18] Charles C. Holt. “Forecasting Seasonals and Trends by Exponentially Weighted Moving Averages”. In: *International Journal of Forecasting* 20.1 (2004), pp. 5–10. DOI: 10.1016/j.ijforecast.2003.09.015.

[19] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. 3rd ed. Melbourne, Australia: OTexts, 2021.

[20] Kendrick Batcharo et al. “The Role of Smart Queue Management Systems in Enhancing Customer Flow and Service Quality”. In: *International Journal for Research Trends and Innovation* 10.9 (2025), b314–b323. URL: https://www.ijrti.org/papers/IJRTI2509140.pdf.

[21] Yuanyang Zou, Renhuai Liu, and Junxia Yuan. “Optimizing Customer Satisfaction in a Finite Service Time Queueing System with Incentive-based Queueing Length for Service Rates”. In: *Journal of Computational and Applied Mathematics* 485 (2026), p. 117510. DOI: 10.1016/j.cam.2026.117510.

[22] Tamrat Yifter et al. “Modeling and Simulation of Queuing System to Improve Service Quality at Commercial Bank of Ethiopia”. In: *Cogent Engineering* 10.2 (2023), p. 2274522. DOI: 10.1080/23311916.2023.2274522.

[23] Arya Nair et al. “A Survey on Queue Management Systems”. In: *International Journal for Research Trends and Innovation* 10.1 (2025), a186–a192. URL: https://www.ijrti.org/papers/IJRTI2501027.pdf.

[24] Hummy Song, Mor Armony, and Guillaume Roels. “Queue Configurations and Operational Performance: An Interplay Between Customer Ownership and Queue Length Awareness”. In: *Manufacturing & Service Operations Management* 26.6 (2024), pp. 2284–2304. DOI: 10.1287/msom.2023.0202.

[25] N. N. A. Rashid et al. “A Conceptual Design of Smart Queueing Management System for Multiple Organizations in Urban Transformation Centre (UTC) Melaka”. In: *AIP Conference Proceedings*. Vol. 2544. 2023, p. 030015. DOI: 10.1063/5.0116320.

[26] Marc Lester Mallari et al. "CLIQUE: A Web-Based Queue Management System with Real-Time Queue Tracking and Notification of Units for Angeles University Foundation Office of the University Registrar". In: *Proceedings of the 12th Annual International Conference on Industrial Engineering and Operations Management*. Istanbul, Turkey: IEOM Society International, 2022. DOI: 10.46254/AN12.20220359.

[27] John D. C. Little and Stephen C. Graves. "Little's Law". In: *Building Intuition: Insights from Basic Operations Management Models and Principles*. Ed. by Dilip Chhajed and Timothy J. Lowe. Vol. 115. International Series in Operations Research & Management Science. New York: Springer, 2008, pp. 81–100. DOI: 10.1007/978-0-387-73699-0_5.

[28] *Real-Time People Counting System Using YOLOv8 Object Detection*. IEEE Conference Publication, 2024. 2024. URL: https://ieeexplore.ieee.org/document/10419684.

[29] *Person Detection Using Proposed YOLOv8 Algorithm with DeepSORT Tracking*. IEEE Conference Publication, 2025. 2025. URL: https://ieeexplore.ieee.org/document/10967648/.

[30] *Deep Learning-Based Multi-Object Tracking: A Comprehensive Survey*. Adzemovic, M. (2025). arXiv:2506.13457. 2025. arXiv: 2506.13457. URL: https://arxiv.org/pdf/2506.13457.

[31] *Deep Learning Based Occluded Person Re-Identification: A Survey (ACM TOMM)*. Peng, Y. et al. (2023). ACM Transactions on Multimedia Computing, 20(3). 2023. URL: https://dl.acm.org/doi/10.1145/3610534.

[32] *Occluded Person Re-Identification with Deep Learning: A Survey and Perspectives*. Ning, C. et al. (2023). arXiv:2311.00603 / ScienceDirect. 2023. arXiv: 2311.00603. URL: https://arxiv.org/abs/2311.00603.

[33] *A Real-Time Queue Tracking Method for Waiting Time Estimation*. IEEE Conference Publication, 2022. 2022. URL: https://ieeexplore.ieee.org/document/9864749/.

[34] *Waiting Time Prediction in Queue Management: Leveraging Machine Learning Approach*. IEEE Conference Publication, 2025. 2025. URL: https://ieeexplore.ieee.org/document/10955915/.

[35] *Exponential Smoothing Model Selection for Forecasting*. International Journal of Forecasting, Volume 39, Issue 3, 2023. 2023. URL: https://www.sciencedirect.com/science/article/abs/pii/S016920700500107X.

[36] *Review of Methods and Models for Forecasting Electricity Consumption*. Energies, 18(15), 4032, 2025. MDPI. 2025. URL: https://www.mdpi.com/1996-1073/18/15/4032.

[37] *Real-Time Prediction of Delay Distribution in Service Systems Using Mixture Density Networks*. arXiv:1912.08368 (2020–2021). 1912. arXiv: 1912.08368. URL: https://arxiv.org/pdf/1912.08368.

[38] *Low-Latency Video Anonymization for Crowd Anomaly Detection: Privacy Versus Performance*. IEEE Transactions, 2025. 2025. URL: https://ieeexplore.ieee.org/iel8/10206/11313711/11231379.pdf.

[39] *Evaluating Fairness of Machine Learning Prediction of Prolonged Wait Times in Emergency Department*. Wang, H. et al. (2025). PLOS Digital Health. DOI: 10.1371/journal.pdig.0000751. 2025. URL: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11925291/.

[40] *A Machine Learning Approach to Waiting Time Prediction in Queueing Scenarios*. Kyritsis, A.I. & Deriaz, M. (2020). IEEE AI4I Proceedings. 2020. URL: https://ieeexplore.ieee.org/document/9027796.

[41] *SGST-YOLOv8: An Improved Lightweight YOLOv8 for Real-Time Target Detection for Campus Surveillance*. Cheng, G., Chao, P., Yang, J., & Ding, H. (2024). Applied Sciences, 14(12), 5341. 2024. DOI: 10.3390/app14125341.

[42] *Multi-Object Pedestrian Tracking Using Improved YOLOv8 and OC-SORT*. Xiao, X. & Feng, X. (2023). Sensors, 23(20), 8439. 2023. DOI: 10.3390/s23208439.

[43] *Multi-Objective Pedestrian Tracking Method Based on YOLOv8 and Improved DeepSORT*. Sheng, W. et al. (2024). Mathematical Biosciences and Engineering, 21(2), 1791–1805. 2024. DOI: 10.3934/mbe.2024077.

[44] *Evaluating YOLOv8 and YOLOv11 for Real-Time Person Detection in Crowded Environments*. IEEE Conference Publication, 2025. 2025. URL: https://ieeexplore.ieee.org/document/11088012/.

[45] *A Framework for Pedestrian Attribute Recognition Using Deep Learning*. Applied Sciences, 12(2), 622, 2022. MDPI. 2022. DOI: 10.3390/app12020622.

[46] *Privacy-Preserving Surveillance as an Edge Service Based on Lightweight Video Protection Schemes*. Electronics, 10(3), 236, 2021. MDPI. 2021. DOI: 10.3390/electronics10030236.

[47] *Perception of Risks and Usefulness of Smart Video Surveillance Systems*. Applied Sciences, 12(20), 10435, 2022. MDPI. 2022. DOI: 10.3390/app122010435.

[48] *Automated Enrollment Queueing System in a University Towards Student Experience and Operational Efficiency*. Espinosa, G.S. et al. (2024). ejournals.ph. 2024. URL: https://ejournals.ph/article.php?id=24562.

[49] *Q-Line UCLM: Enhancing Student Service Flow Through Paperless Queue Management*. UC-LM (2025). UCLM Journal of Research and Innovation, Vol. 1 Issue 1. 2025. URL: https://lmportal.uc.edu.ph/storage/research/0801659_[research]_85044d6d.pdf.

[50] *Improving the Security and Reliability of SDN Controller REST APIs Using JSON Web Token (JWT) with OpenID and OAuth 2.0*. IEEE Conference Publication, 2024. 2024. URL: https://ieeexplore.ieee.org/document/10599643/.

[51] *Performance and Security Comparison of JSON Web Tokens (JWT) and PASETO on RESTful APIs*. IEEE Conference Publication, 2023. 2023. URL: https://ieeexplore.ieee.org/document/10277377/.

[52] *Performance Evaluation of a Dynamic RESTful API Using FastAPI, Docker and Nginx*. IEEE Conference Publication, 2025. 2025. URL: https://ieeexplore.ieee.org/document/10881712/.

[53] *An Agile Model-Based Software Engineering Approach Illustrated Through the Development of a Health Technology System*. Software, 2(2), 11, 2023. MDPI. 2023. DOI: 10.3390/software2020011.

[54] *Adoption of Large-Scale Scrum Practices Through the Use of Management 3.0*. Informatics, 9(1), 20, 2022. MDPI. 2022. DOI: 10.3390/informatics9010020.

[55] *People Counting in Public Spaces Using Deep Learning-Based Object Detection and Tracking Techniques*. IEEE Conference Publication, 2023. 2023. URL: https://ieeexplore.ieee.org/document/10183503/.

[56] *Vision-Based People Counting and Tracking for Urban Environments*. Journal of Imaging, 12(1), 27, 2026. MDPI. 2026. DOI: 10.3390/jimaging12010027.

[57] *Deep Learning-Based Video Analysis for Visitor Detection and Tracking in Protected Areas*. Ecological Informatics, 2025. ScienceDirect. 2025. URL: https://www.sciencedirect.com/science/article/pii/S2213078025000362.

[58] *Deep Crowd Anomaly Detection: State-of-the-Art, Challenges, and Future Research Directions*. Artificial Intelligence Review, Springer, 2025. 2025. URL: https://link.springer.com/article/10.1007/s10462-024-11092-8.

[59] *Energy Consumption Forecasts by Gradient Boosting Regression Trees*. Mathematics, 11(5), 1068, 2023. MDPI. 2023. DOI: 10.3390/math11051068.

[60] *Deep Learning-Assisted Mathematical Model for Decongestion Time Prediction at Railroad Grade Crossings*. Neural Computing and Applications, 34, 4715–4732, 2022. 2022. DOI: 10.1007/s00521-021-06625-z.

[61] *Measuring Students' Acceptance and Usability of a Cloud Virtual Desktop Solution for a Programming Course*. Applied Sciences, 11(15), 7157, 2021. MDPI. 2021. DOI: 10.3390/app11157157.

[62] *Perceived Usability Evaluation of Educational Technology Using the PSSUQ: A Systematic Review*. Sustainability, 15(17), 12954, 2023. MDPI. 2023. DOI: 10.3390/su151712954.

[63] *A System Dynamics Model of User Acceptance in Higher Education: Simulating TAM Constructs with Vensim*. IEEE Conference Publication, 2025. 2025. URL: https://ieeexplore.ieee.org/document/11308791/.

[64] *Deep Learning-Based EU-GDPR Compliant Privacy Safeguards in Surveillance Videos*. IEEE Conference Publication, 2022. 2022. URL: https://ieeexplore.ieee.org/abstract/document/9730572/.

[65] *Evaluation and Improvement of the HEI Enrollment Information System in Southern Philippines*. ACM WSSE 2024 Proceedings. 2024. URL: https://dl.acm.org/doi/10.1145/3698062.3698084.

[66] *A Survey on Deep Learning-Based Real-Time Crowd Anomaly Detection for Secure Distributed Video Surveillance*. Personal and Ubiquitous Computing, Springer, 2021. 2021. URL: https://link.springer.com/article/10.1007/s00779-021-01586-5.

[67] *An Enhanced Framework for Real-Time Dense Crowd Abnormal Behavior Detection Using YOLOv8*. Artificial Intelligence Review, Springer, 2025. 2025. URL: https://link.springer.com/article/10.1007/s10462-025-11206-w.

[68] *Disaggregated Retail Forecasting: A Gradient Boosting Approach*. Applied Soft Computing, Volume 141, 2023. ScienceDirect. 2023. URL: https://www.sciencedirect.com/science/article/abs/pii/S1568494623003010.

[69] *Design and Content Validation Using Expert Opinions of an Instrument: The 'PONTE A 100' Questionnaire*. Healthcare, 11(14), 2038, 2023. MDPI. 2038. DOI: 10.3390/healthcare11142038.

[70] *Non-Maximum Suppression Performs Later in Multi-Object Tracking*. Applied Sciences, 12(7), 3334, 2022. MDPI. 2022. DOI: 10.3390/app12073334.

[71] *Multi-Attribute NMS: An Enhanced NMS for Pedestrian Detection in Crowded Scenes*. Applied Sciences, 13(14), 8073, 2023. MDPI. 2023. DOI: 10.3390/app13148073.

[72] *An Effectively Finite-Tailed Updating for Multiple Object Tracking in Crowd Scenes*. Applied Sciences, 12(3), 1061, 2022. MDPI. 2022. DOI: 10.3390/app12031061.

[73] *Multi-Object Tracking with Predictive Information Fusion and Adaptive Measurement Noise*. Applied Sciences, 15(2), 736, 2025. MDPI. 2025. DOI: 10.3390/app15020736.

[74] *Dissatisfaction-Considered Waiting Time Prediction for Outpatients with Interpretable Machine Learning*. Health Care Management Science, 2024. DOI: 10.1007/s10729-024-09676-5. 2024. URL: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11461612/.

[75] *On Little's Formula in Multiphase Queues*. Mathematics, 9(18), 2282, 2021. MDPI. 2021. DOI: 10.3390/math9182282.

[76] *Analysis of Multi-Server Queueing System with Flexible Priorities*. Mathematics, 11(4), 1040, 2023. MDPI. 2023. DOI: 10.3390/math11041040.

[77] *Design and Validation of a Questionnaire on Influence of the University Classroom on Motivation and Sociability*. Education Sciences, 11(4), 183, 2021. MDPI. 2021. DOI: 10.3390/educsci11040183.

[78] *Test–Retest Reliability and Internal Consistency of a Newly Developed Questionnaire*. International Journal of Environmental Research and Public Health, 20(5), 4407, 2023. MDPI. 2023. DOI: 10.3390/ijerph20054407.

[79] *Efficient Public Transport System and Digital Ticketing*. IEEE Conference Publication, 2023. 2023. URL: https://ieeexplore.ieee.org/document/10151493/.

[80] *DetTrack: An Algorithm for Multiple Object Tracking by Improving Occlusion Object Detection*. Electronics, 13(1), 91, 2024. MDPI. 2024. DOI: 10.3390/electronics13010091.

[81] *Fusing Appearance and Spatio-Temporal Models for Person Re-Identification and Tracking*. PMC / MDPI, 2021. 2021. URL: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8321031/.

[82] *A Parallel Patient Treatment Time Prediction Algorithm and Its Applications in Hospital Queuing-Recommendation*. IEEE Transactions on Services Computing. 2025. URL: https://ieeexplore.ieee.org/document/7458801/.

[83] *Design and Implementation of Environmental Monitoring System Using Flask-Based Web Application*. Engineering Proceedings, 92(1), 37, 2025. MDPI. 2025. DOI: 10.3390/engproc2025092037.

[84] *Adaptive Kalman Filter for Real-Time Visual Object Tracking Based on Autocovariance Least Square Estimation*. Li, J. et al. (2024). Applied Sciences, 14(3), 1045. 2024. DOI: 10.3390/app14031045.

[85] *Achieving Adaptive Visual Multi-Object Tracking with Unscented Kalman Filter*. Sensors, 22(23), 9106, 2022. MDPI. 2022. DOI: 10.3390/s22239106.

[86] *Robust Visual Tracking with Reliable Object Information and Kalman Filter*. Sensors, 21(3), 889, 2021. MDPI. 2021. DOI: 10.3390/s21030889.

[87] *Bounding Box Dynamics in Visual Tracking: Modeling and Noise Covariance Estimation*. IEEE Conference Publication, 2023. 2023. URL: https://ieeexplore.ieee.org/document/10224186/.

[88] *A Systematic Review of Accessibility Techniques for Online Platforms: Current Trends and Challenges*. Applied Sciences, 14(22), 10337, 2024. MDPI. 2024. DOI: 10.3390/app142210337.

[89] *Design and Implementation of a Voice-Enabled Web Application for Customer Interaction Using NLP*. IEEE Conference Publication, 2024. 2024. URL: https://ieeexplore.ieee.org/document/10739046/.

[90] *Self-Service System with Rating-Dependent Arrivals*. Mathematics, 10(3), 297, 2022. MDPI. 2022. DOI: 10.3390/math10030297.

[91] *Dimensioning the Optimal Number of Parallel Service Desks in the Passenger Handling Process at Airports*. Jencova, E. et al. (2023). Aerospace, 10(1), 50. 2023. DOI: 10.3390/aerospace10010050.

[92] *Machine Learning-Based Prediction of Passenger Waiting Time During Check-in Process*. Journal of Systems Science and Systems Engineering, Springer, 2025. 2025. URL: https://link.springer.com/article/10.1007/s11518-025-5665-9.

[93] *Predictive Sales and Operations Planning Based on Statistical Treatment of Demand*. Applied Sciences, 11(1), 233, 2021. MDPI. 2021. DOI: 10.3390/app11010233.

[94] *Predictive Analytics for Sales Demand Forecasting Using HML Model (LSTM + XGBoost)*. IEEE Conference Publication, 2025. 2025. URL: https://ieeexplore.ieee.org/document/11026743.

[95] *Effective Multi-Object Tracking via Global Object Models and Object Constraint Learning*. Applied Sciences, 2022. MDPI / PMC. 2022. URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC9609386/.

[96] *Unveiling Personnel Movement in a Larger Indoor Area with a Non-Overlapping Multi-Camera System*. arXiv:2104.04662, 2021. 2021. arXiv: 2104.04662. URL: https://arxiv.org/pdf/2104.04662.

[97] *AI Chatbot Using Web Speech API and Node.js*. IEEE Conference Publication, 2022. 2022. URL: https://ieeexplore.ieee.org/document/9760803/.

[98] *Conversion of Videoconference Speech into Text Based on WebRTC and Web Speech APIs*. IEEE Conference Publication, 2021. 2021. URL: https://ieeexplore.ieee.org/document/9513968/.

[99] *3D-DIoU: 3D Distance Intersection over Union for Multi-Object Tracking in Point Cloud*. Sensors, 23(7), 3390, 2023. MDPI. 2023. DOI: 10.3390/s23073390.

[100] *Smart Wildlife Monitoring: Real-Time Hybrid Tracking Using Kalman Filter and Local Binary Similarity with HSV + Pearson Correlation*. Computers, 14(8), 307, 2025. MDPI. 2025. DOI: 10.3390/computers14080307.

[101] *Person Re-Identification by Discriminative Local Features of Overlapping Stripes*. Symmetry, 12(4), 647, 2020. MDPI. 2020. DOI: 10.3390/sym12040647.

[102] *Cross-Modality Person Re-Identification Based on Heterogeneous Center Loss and Non-Local Features*. Entropy, 23(7), 919, 2021. MDPI. 2021. DOI: 10.3390/e23070919.

[103] *Classification of Barriers to Digital Transformation in Higher Education Institutions: Systematic Literature Review*. Education Sciences, 13(7), 746, 2023. MDPI. 2023. DOI: 10.3390/educsci13070746.

[104] *Digital Learning and Digital Institution in Higher Education*. Education Sciences, 13(1), 88, 2023. MDPI. 2023. DOI: 10.3390/educsci13010088.

[105] *Digital Transformation in Higher Education Institutions: A Systematic Literature Review*. Sensors, 20(11), 3291, 2020. MDPI. 2020. DOI: 10.3390/s20113291.

[106] *The Automation of Client Servicing in University and College Admission Offices*. IEEE Conference Publication, 2023. 2023. URL: https://ieeexplore.ieee.org/document/10092103/.

[107] *Supporting Educational Administration via Emergent Technologies*. Education Sciences, 16(1), 29, 2026. MDPI. 2026. DOI: 10.3390/educsci16010029.

[108] *The Technology Interface and Student Engagement Are Significant Stimuli in Sustainable Student Satisfaction*. Sustainability, 15(10), 7923, 2023. MDPI. 2023. DOI: 10.3390/su15107923.

[109] *Enhanced NMS for Detection of Steel Surface Defects (Soft-NMS Application)*. Mathematics, 11(18), 3898, 2023. MDPI. 2023. DOI: 10.3390/math11183898.

[110] *Predictive Analytics for Demand Forecasting (ARIMA/MAPE context)*. Applied Sciences, 15(2), 770, 2025. MDPI. 2025. DOI: 10.3390/app15020770.

[111] *Person Re-Identification Enhanced by Super-Resolution Technology*. Electronics, 14(23), 4647, 2025. MDPI. 2025. DOI: 10.3390/electronics14234647.

[112] *Spatial Non-Maximum Suppression for Object Detection Using Correlation and Dynamic Thresholds*. IEEE Conference Publication, 2021. 2021. URL: https://ieeexplore.ieee.org/document/9614023/.

[113] *Adaptive IoU Thresholding for Improving Small Object Detection*. Diagnostics, 13(1), 104, 2023. MDPI. 2023. DOI: 10.3390/diagnostics13010104.

[114] *An Improved YOLOv8-Based Lightweight Attention Mechanism for Cross-Scale Feature Fusion*. Liu, S. et al. (2025). Remote Sensing, 17(6), 1044. 2025. DOI: 10.3390/rs17061044.

[115] *YOLOv8: A Novel Object Detection Algorithm with Enhanced Performance and Robustness*. Varghese, R. & Sambath, M. (2024). 2024 ADICS, IEEE, pp. 1–6. 2024. URL: https://ieeexplore.ieee.org/document/10612459/.

[116] *The Coefficient of Determination R-Squared Is More Informative Than SMAPE, MAE, MAPE, MSE and RMSE*. Chicco, D., Warrens, M.J., & Jurman, G. (2021). PeerJ Computer Science. 2021. DOI: 10.7717/peerj-cs.623.

[117] *Comparative Evaluation of Model Accuracy Using MAE, MSE, and MAPE in Agile Project Management*. Mathematics, 12(16), 2529, 2024. MDPI. 2024. DOI: 10.3390/math12162529.

[118] *Evaluation Metrics for Wind Power Forecasts: A Comprehensive Review and Statistical Analysis*. Energies, 15(24), 9657, 2022. MDPI. 2022. DOI: 10.3390/en15249657.

[119] *Performance Evaluation of Machine Learning Models for Predicting Energy Consumption and Occupant Dissatisfaction*. Buildings, 15(1), 39, 2025. MDPI. 2025. DOI: 10.3390/buildings15010039.

[120] *An Active Multi-Object Ultrafast Tracking System with CNN-Based Hybrid Object Detection*. Sensors, 23(8), 4150, 2023. MDPI. 2023. DOI: 10.3390/s23084150.

[121] *Object Detection and Tracking with YOLO and the Sliding Innovation Filter*. Moksyakov, A. et al. (2024). Sensors, 24(7), 2107. MDPI. 2024. DOI: 10.3390/s24072107.

[122] *A Real-Time Wrong-Way Vehicle Detection Based on YOLO and Centroid Tracking*. arXiv:2210.10226, 2022. 2022. arXiv: 2210.10226. URL: https://arxiv.org/pdf/2210.10226.

[123] *A Comparative Analysis of Machine Learning Algorithms for Predicting Student Academic Success (MAE/XGBoost)*. Applied Sciences, 15(11), 5879, 2025. MDPI. 2025. DOI: 10.3390/app15115879.